Contents

1 Getting started . . . **5**
- 1.1 Install . . . 5
- 1.2 Your first spec . . . 5
- 1.3 What just happened . . . 6
- 1.4 Introducing a bug . . . 6
- 1.5 Fix it . . . 7
- 1.6 Key commands . . . 7
- 1.7 Choosing a backend . . . 8
- 1.8 The full Spectre workflow . . . 8
- 1.9 What's next . . . 9

2 The Spectre language . . . **9**
- 2.1 Variables . . . 9
- 2.2 Collections . . . 10
- 2.3 Enumerations . . . 11
- 2.4 Parameters . . . 12
- 2.5 Init block . . . 12
- 2.6 Actions . . . 12
- 2.7 Invariants . . . 13
- 2.8 The complete bank account spec . . . 14
- 2.9 Temporal properties . . . 15
- 2.10 Descriptions . . . 15
- 2.11 Quick-reference card . . . 15
- 2.12 What's next . . . 16

3 Safety properties and invariants . . . **16**
- 3.1 What a safety property says . . . 16
- 3.2 Three patterns for invariants . . . 16
- 3.3 A spec with a bug . . . 17
- 3.4 Reading a violation trace . . . 17
- 3.5 Longer traces . . . 18
- 3.6 Using `--suggest-repairs` . . . 19
- 3.7 Checking multiple invariants . . . 19
- 3.8 Parameterized safety . . . 19
- 3.9 State space size and `--max-states` . . . 20
- 3.10 Invariant-writing checklist . . . 20
- 3.11 Workflow summary . . . 20

4 Temporal properties and liveness . . . **21**
- 4.1 Safety vs liveness . . . 21
- 4.2 `always` — a condition that holds forever . . . 21
- 4.3 `eventually` — something must happen . . . 22
- 4.4 `until` — hold one condition until another . . . 23
- 4.5 Leads-to — if P then eventually Q . . . 23
- 4.6 Combining temporal properties . . . 24
- 4.7 Temporal verification with large state spaces . . . 25

4.8 Quick-reference . . . 25

5 Fairness — WF and SF . . . 25
5.1 Why fairness matters . . . 25
5.2 Weak fairness: WF(action) . . . 26
5.3 Strong fairness: SF(action) . . . 27
5.4 WF and SF in the same spec . . . 28
5.5 BFS for learning, TLC for validation . . . 28
5.6 Fairness checklist . . . 28

6 Concurrent and distributed systems . . . 29
6.1 What's different about concurrent specs . . . 29
6.2 Start with a broken mutex . . . 29
6.3 Fix it with a lock . . . 29
6.4 Producer-consumer with a bounded buffer . . . 30
6.5 Distributed message queue . . . 31
6.6 Good invariants for concurrent specs . . . 32
6.7 When the state space gets large . . . 32
6.8 Concurrency workflow . . . 32

7 Modules, parameters, and reuse . . . 33
7.1 Constants — changing the spec without editing it . . . 33
7.2 Modules — reusable spec components . . . 34
7.3 Module inheritance — parameterized reuse . . . 34
7.4 Splitting a large spec into modules . . . 35
7.5 Workflow: from flat to modular . . . 36
7.6 Constant best practices . . . 37

8 Collections and distributed specifications . . . 37
8.1 The three collection types . . . 37
8.2 Set operations . . . 37
8.3 Map operations . . . 38
8.4 List operations . . . 38
8.5 Example: Two-Phase Commit . . . 39
8.6 Why collection specs find bugs scalars can't . . . 41
8.7 State-space note: collections multiply states . . . 41
8.8 Collection syntax quick-reference . . . 41

9 Reading counterexamples and debugging . . . 42
9.1 Anatomy of a violation trace . . . 42
9.2 Five-step debugging workflow . . . 43
9.3 Worked example: the transfer bug . . . 43
9.4 Using `--suggest-repairs` . . . 44
9.5 Multiple violations . . . 45
9.6 Dealing with bounded results . . . 45
9.7 Common patterns and their fixes . . . 46
9.8 Tips for writing debuggable specs . . . 46

10 Choosing a verification backend . . . 46

10.1 Two tracks: learning vs. production . . . 46
10.2 The four backends at a glance . . . 47
10.3 Quick decision guide . . . 47
10.4 BFS backend (default — learning mode) . . . 48
10.5 TLC backend . . . 48
10.6 Apalache backend . . . 49
10.7 Z3 backend (bounded model checking) . . . 50
10.8 Comparison table . . . 50
10.9 Progressive verification workflow . . . 51
10.10Installing the backends . . . 51

11 Mining specs from Rust . . . 51
11.1 What mining does . . . 52
11.2 Your first mine . . . 52
11.3 What gets extracted vs. what gets skipped . . . 53
11.4 Seeing what was skipped: `--show-omissions` . . . 54
11.5 Evaluate a whole directory . . . 54
11.6 Enrich the mined spec . . . 55
11.7 LLM-enhanced mining . . . 56
11.8 The full mining workflow . . . 56

12 Drift detection and incremental re-verification . . . 56
12.1 The drift problem . . . 56
12.2 Running sync . . . 57
12.3 CI integration . . . 57
12.4 Incremental re-verification . . . 58
12.5 Impact analysis . . . 58
12.6 Full sync workflow . . . 59
12.7 Sync evaluation: what it checks . . . 59
12.8 Tips for keeping sync useful . . . 60

13 Model-based testing . . . 60
13.1 The idea . . . 60
13.2 Generate a trace . . . 60
13.3 Generate the driver . . . 61
13.4 CI-ready driver with `--ci` . . . 61
13.5 Run the replay . . . 62
13.6 Violation traces as regression tests . . . 62
13.7 Repair suggestions before fixing . . . 63
13.8 Checking refinement . . . 63
13.9 Complete MBT workflow . . . 63

14 Runtime monitoring . . . 64
14.1 The gap formal verification leaves . . . 64
14.2 What the generated monitor actually is . . . 65
14.3 Generate a monitor . . . 65
14.4 Wire it into your application . . . 66
14.5 Handle violations your way . . . 66

14.6 Validate monitor traces before deploying . . . 66
14.7 What the monitor checks — and what it doesn't . . . 67
14.8 Full monitoring workflow . . . 67
14.9 Performance overhead . . . 67

15 Benchmarking and performance . . . 68
15.1 Run the benchmarks . . . 68
15.2 Understand state-space growth . . . 68
15.3 State-space growth diagram . . . 69
15.4 Bounded exploration options . . . 69
15.5 Partial-order reduction . . . 69
15.6 Property-guided exploration . . . 70
15.7 When to switch backends . . . 70
15.8 Progressive verification strategy . . . 70
15.9 Benchmark in CI . . . 71

16 TLA+ and Apalache backend . . . 71
16.1 Quick start . . . 71
16.2 How Spectre translates to TLA+ . . . 72
16.3 Expression mapping reference . . . 73
16.4 Why TLC for fairness? . . . 73
16.5 TLC vs Apalache: when to use which . . . 74
16.6 Pass parameters to TLC . . . 74
16.7 Read TLC output . . . 74
16.8 TLC installation . . . 75
16.9 Apalache installation . . . 75

17 Z3 bounded model checking . . . 75
17.1 Quick start . . . 75
17.2 How bounded model checking works . . . 76
17.3 SMT-LIB2 encoding example . . . 76
17.4 Read Z3 output . . . 77
17.5 When to use Z3 BMC . . . 78
17.6 Choose the right depth . . . 78
17.7 Z3 vs other backends . . . 78
17.8 Progressive verification with Z3 . . . 79
17.9 Z3 installation . . . 79

18 Appendix A: Language Grammar (EBNF) . . . 80
18.1 A.1 Top-level structure . . . 80
18.2 A.2 Variable and constant declarations . . . 81
18.3 A.3 Types . . . 81
18.4 A.4 Enum declarations . . . 81
18.5 A.5 Type aliases . . . 82
18.6 A.6 Init declaration . . . 82
18.7 A.7 Action declarations . . . 82
18.8 A.8 Statements . . . 83
18.9 A.9 Function declarations . . . 83

18.10A.10 Invariant and temporal declarations . . . 84
18.11A.11 Expressions . . . 84
18.12A.12 If, let, and lambda expressions . . . 84
18.13A.13 Collection literals . . . 85
18.14A.14 Temporal expressions . . . 85
18.15A.15 Module system . . . 86
18.16A.16 Operator precedence . . . 86
18.17A.17 Comments . . . 87

19 Appendix B: Evaluation Order and Priming Rules . . . 87
19.1 B.1 Simultaneous assignment . . . 87
19.2 B.2 Implicit UNCHANGED . . . 88
19.3 B.3 Double assignment is an error . . . 88
19.4 B.4 Guards evaluate in the pre-action state . . . 88
19.5 B.5 Init is also simultaneous . . . 89
19.6 B.6 Actions are atomic . . . 89
19.7 B.7 Expression evaluation . . . 89
19.8 B.8 Priming summary . . . 91

1 Getting started

1.1 Install

```
# macOS / Linux - build from source
git clone https://github.com/BoundlessProducts/spectre
cd spectre
go build -o spectre ./cmd/spectre
sudo mv spectre /usr/local/bin/

spectre --version
# spectre version dev
```

No runtime dependencies, no Docker, no JVM needed for the default BFS backend.

1.2 Your first spec

Save this as `counter.spec`:

```
var value: int
const MAX: int = 5

init { value = 0 }

action increment {
  require value < MAX
  value' = value + 1
```

```
}

action reset {
  value' = 0
}

invariant non_negative  { value >= 0 }
invariant within_bounds { value <= MAX }
```

Run it:

```
spectre verify counter.spec
```

```
Exhaustive verification: traversed 6 states (full reachable space)
Found no violations.
```

1.3 What just happened

```
counter.spec
  +-- var value: int              <- one piece of state
  +-- const MAX: int = 5          <- a named constant
  +-- init { value = 0 }          <- where to start
  +-- action increment { ... }    <- one possible transition
  +-- action reset { ... }        <- another transition
  +-- invariant within_bounds ... <- the property to check
          |
          v
    spectre verify
          |
    BFS explores all 6 reachable states
    {0, 1, 2, 3, 4, 5}
          |
    checks both invariants in every state
          |
          v
        (ok) safe
```

Spectre uses **breadth-first search (BFS)** as its default learning and exploration backend — zero setup, instant feedback, great for writing and debugging specs. It finds every state reachable from `init` and checks your invariants in all of them. Six states here because `value` can be `0`--`5` and `reset` can always bring it back.

BFS is perfect for learning Spectre and iterating quickly. For production liveness verification or large state spaces, use `--backend tlc` or `--backend apalache` (see the Backend Selection chapter).

1.4 Introducing a bug

Remove the `require` from `increment`:

```
action increment {
  // require value < MAX   <- commented out
  value' = value + 1
}
```

```
spectre verify counter.spec
```

```
Violations:

1.   Invariant violated: within_bounds
  Action 'increment' would violate invariants: [invariant within_bounds violated]
  Counterexample trace (5 step(s)):
    [init]
    [step 1] increment
      value: 0 -> 1
    [step 2] increment
      value: 1 -> 2
    [step 3] increment
      value: 2 -> 3
    [step 4] increment
      value: 3 -> 4
    [step 5] increment
      value: 4 -> 5
    [VIOLATION] invariant "within_bounds" fails in this state
      value = 5
```

Spectre shows the state where the *next* `increment` would violate `within_bounds` (value $5 + 1 = 6 > \text{MAX}$). Add back `require value < MAX` to fix it.

Spectre found the shortest path to a bad state: six increments from zero.

1.5 Fix it

Restore `require value < MAX`. This tells Spectre: "increment can only fire when `value < MAX`." If the condition is false the action is disabled — not an error, the system just can't take that step from that state.

1.6 Key commands

Command	What it does
`spectre verify <file>`	BFS — exhaustively checks all reachable states (learning/exploration mode)
`spectre verify <file> --backend z3`	Bounded check — fast, finds shallow bugs
`spectre verify <file> --backend tlc`	Delegates to TLA+ Tools (validated fairness)

Command	What it does
`spectre verify <file> --backend apalache`	Delegates to Apalache (symbolic, large specs)
`spectre mine --lang rust <file.rs>`	Extract a spec skeleton from Rust source
`spectre simulate <file> --steps 20`	Random walk through the state space
`spectre benchmark`	Time BFS across a set of spec files

1.7 Choosing a backend

```
      Your goal
          |
          v
  +-----------------------------------------------------+
  | Learning / exploring a new spec?                    |
  |   -> spectre verify  (default BFS, zero setup)      |
  +-----------------------------------------------------+
  | Find shallow bugs quickly (<=20 steps)?             |
  |   -> --backend z3 --bmc-depth 20                    |
  +-----------------------------------------------------+
  | Exhaustive proof on a small-medium spec?            |
  |   -> spectre verify  (default BFS)                  |
  +-----------------------------------------------------+
  | Liveness / fairness properties you can trust?       |
  |   -> --backend tlc  or  --backend apalache          |
  +-----------------------------------------------------+
  | Spec too large for BFS (millions of states)?        |
  |   -> --backend apalache  (symbolic, handles large)  |
  +-----------------------------------------------------+
```

The Backend Selection chapter covers the full decision guide.

1.8 The full Spectre workflow

```
Rust source
    |
    v
spectre mine --lang rust src/account.rs
    |
    v
account.spec  <- skeleton; add invariants by hand
    |
    v
```

```
spectre verify account.spec
    |
    +-- (ok) safe -----------------> commit .spec alongside source
    |
    +-- (x) violation
              |
              v
         spectre verify account.spec --suggest-repairs
              |
              v
         CANDIDATE patches  <- review, apply, re-verify
```

Once safe, one spec drives three downstream workflows:

```
account.spec
    |
    +---> spectre generate-driver  ->  Rust conformance test harness  (Model-Based Testing chap
    +---> spectre generate-monitor ->  embedded runtime monitor       (Runtime Monitoring chapt
    +---> spectre sync account.rs  ->  drift detection as code evolves (Drift and Sync chapter)
```

1.9 What's next

- **The Spectre Language chapter** — every language construct with examples
- **Safety Properties chapter** — safety invariants in depth, reading violations, repair suggestions
- **Backend Selection chapter** — picking the right backend for your problem

2 The Spectre language

2.1 Variables

You declare state with `var`. Four primitive types:

Type	Declaration	Default
`int`	`var count: int`	`0`
`bool`	`var active: bool`	`false`
`str`	`var label: str`	`""`
`float`	`var ratio: float`	`0.0`

```
var count:  int
var active: bool
var label:  str
var ratio:  float
```

Variables live for the entire run — there are no local variables, only state.

2.2 Collections

Spectre supports first-class collection types for modelling sets of nodes, queues, and mappings:

Type	Declaration	Example init
`Set<T>`	`var nodes: Set<int>`	`{1, 2, 3}` or `{}`
`Map<K,V>`	`var votes: Map<int, bool>`	`{}`
`List<T>`	`var queue: List<int>`	`[]`

Common collection methods:

Method / operator	Works on	Returns
`x in S`	Set, Map	bool — membership test (infix)
`.contains(x)`	Set, Map	bool — same, method form
`.union(other)`	Set	Set — combined elements
`.intersection(other)`	Set	Set — common elements
`.difference(other)`	Set	Set — elements in receiver but not argument
`.size()`	Set, Map, List	int
`.put(k, v)`	Map	Map — with key set
`.get(k)`	Map	value
`.append(x)`	List	List — with element added
`.head()`	List	first element
`.tail()`	List	List without first element
`.filter(fn)`	Set, List	filtered collection
`.forall(fn)`	Set, List	bool — all match
`.exists(fn)`	Set, List	bool — any match

A working example — a simple voting spec with a set of voters:

```
const QUORUM: int = 2

var voted:    Set<int>   // ids of nodes that have voted
var approved: bool

init {
  voted    = {}
```

```
  approved = false
}

action cast_vote {
  require voted.size() < 3        // at most 3 voters
  voted' = voted.union({voted.size() + 1})
}

action approve {
  require voted.size() >= QUORUM
  require !approved
  approved' = true
}

invariant no_approval_without_quorum {
  !approved || voted.size() >= QUORUM
}
```

```
spectre verify vote.spec
# Exhaustive verification: traversed 6 states (full reachable space)
# Found no violations.
```

The `in` operator is the most natural way to check membership inside `require` and `invariant` expressions:

```
require !(node_id in voted)   // has not voted yet
invariant inv { x in S || x > 0 }
```

See the **Collections and distributed specifications** chapter for the full Two-Phase Commit example using `Set`, `Map`, and `in` together.

2.3 Enumerations

Define a named set of values with `enum`:

```
enum TrafficLight { Red, Yellow, Green }
var signal: TrafficLight
```

A complete traffic-light spec:

```
enum TrafficLight { Red, Yellow, Green }

var signal: TrafficLight

init { signal = TrafficLight.Red }

action go     { require signal == TrafficLight.Red     signal' = TrafficLight.Green  }
action caution { require signal == TrafficLight.Green  signal' = TrafficLight.Yellow }
action stop   { require signal == TrafficLight.Yellow  signal' = TrafficLight.Red    }
```

```
invariant only_valid_transitions {
  signal == TrafficLight.Red || signal == TrafficLight.Yellow || signal == TrafficLight.Green
}

spectre verify traffic.spec
# Exhaustive verification: traversed 3 states (full reachable space)
# Found no violations.
```

2.4 Parameters

Constants are named values that don't change during a run:

```
const CAPACITY:  int = 10
const THRESHOLD: int = 7
```

- Override on the command line: `spectre verify queue.spec --param CAPACITY=20`
- Sweep a range: `spectre verify queue.spec --param-range CAPACITY=1..20`
- Use in invariants: `invariant not_full { items <= CAPACITY }`

2.5 Init block

```
init {
  balance = 0
  active  = true
  phase   = Phase.Idle
}
```

- Every variable you list is set to that value.
- Variables not listed get their type's zero value: `0`, `false`, `""`, first enum variant.
- You can list variables in any order.

2.6 Actions

An action describes one atomic transition of the system:

```
action deposit {           <- name
  require status == Status.Active  <- precondition (action disabled if false)
  require amount > 0               <- multiple requires are AND'd together
  balance' = balance + amount      <- next-state value (prime notation)
  count'   = count + 1             <- all assignments apply simultaneously
}
```

Key rules:

- `x'` means "the value of `x` in the next state."
- An action with a failing `require` is simply **disabled** — it's not an error.
- All assignments in one action are **simultaneous** — `x' = y` and `y' = x` swaps them correctly.
- Variables not assigned by an action **keep their current value** automatically.

2.6.1 Example 1: simple counter

```
var value: int
const MAX: int = 10

action increment {
  require value < MAX
  value' = value + 1
}

action decrement {
  require value > 0
  value' = value - 1
}
```

2.6.2 Example 2: bank deposit with two requires

```
var balance: int
const LIMIT: int = 1000

action deposit {
  require balance < LIMIT
  require balance + 100 <= LIMIT
  balance' = balance + 100
}
```

2.6.3 Example 3: enum state machine transition

```
enum Phase { Idle, Running, Done }
var phase: Phase
var started: bool

action start {
  require phase == Phase.Idle
  phase'   = Phase.Running
  started' = true
}

action finish {
  require phase == Phase.Running
  phase' = Phase.Done
}
```

2.7 Invariants

You're telling Spectre: "this must be true in every reachable state."

```
invariant solvent    { balance >= 0 }
invariant within_cap { balance <= LIMIT }
```

- Spectre checks every invariant after every action in every reachable state.
- A violation = one reachable state where the condition is false.
- You can have as many invariants as you want; all are checked independently.

2.8 The complete bank account spec

Everything above pulled together into one file:

```
// bank.spec

enum Status { Active, Frozen }

var balance: int
var status:  Status
const LIMIT: int = 1000

init {
  balance = 0
  status  = Status.Active
}

action deposit {
  require status == Status.Active
  require balance + 100 <= LIMIT
  balance' = balance + 100
}

action withdraw {
  require status == Status.Active
  require balance >= 100
  balance' = balance - 100
}

action freeze   { status' = Status.Frozen }
action unfreeze { status' = Status.Active }

invariant solvent     { balance >= 0 }
invariant within_cap  { balance <= LIMIT }
```

```
spectre verify bank.spec
```

```
Exhaustive verification: traversed 22 states (full reachable space)
Found no violations.
```

2.9 Temporal properties

Temporal properties let you express liveness — things that must *eventually* happen, or *always* remain true over infinite runs:

```
temporal always_solvent  { always(balance >= 0) }
temporal progress        { eventually(balance > 0) }
temporal freeze_recovers { always(status == Status.Frozen -> eventually(status == Status.Active
```

Keyword	Meaning
`always(P)`	P holds in every state of every run
`eventually(P)`	P holds in some state of every run
`P until Q`	P holds until Q becomes true
`WF(action)`	Weak fairness: if `action` is always enabled, it eventually fires
`SF(action)`	Strong fairness: if `action` is infinitely often enabled, it fires infinitely often

BFS is the learning backend — great for exploring what your spec does. For **fairness** (WF/SF) and liveness properties you need to trust, use `--backend tlc` or `--backend apalache`. The Fairness chapter covers this in detail.

2.10 Descriptions

Add a human-readable description to any action:

```
description "Deposits 100 units - requires account active and not at cap"
action deposit {
  require status == Status.Active
  require balance + 100 <= LIMIT
  balance' = balance + 100
}
```

These appear in violation traces and generated code.

2.11 Quick-reference card

Keyword	Purpose
`var`	Declare a state variable
`const`	Declare a named constant (override with `--param`)
`enum`	Declare an enumeration
`init`	Set initial values
`action`	Declare a state transition
`require`	Precondition for an action
`invariant`	Safety property (checked in every state)

Keyword	Purpose
`temporal`	Liveness / fairness property
`always`	Temporal: holds in every state
`eventually`	Temporal: holds in some future state
`until`	Temporal: P holds until Q
`WF`	Weak fairness over an action
`SF`	Strong fairness over an action
`x in S`	Membership test: true if x is in Set or Map S
`description`	Human-readable label for an action

2.12 What's next

- **Safety Properties chapter** — safety properties in depth: writing good invariants, reading violation traces
- **Temporal Properties chapter** — temporal properties: liveness and fairness with real examples

3 Safety properties and invariants

3.1 What a safety property says

"Nothing bad ever happens."

- An `invariant` is a boolean expression that must hold in **every reachable state**.
- A violation = one state where it doesn't hold.
- Spectre finds the **shortest path** to that state (BFS guarantee).

3.2 Three patterns for invariants

3.2.1 1. Resource bound

```
var items:  int
const CAPACITY: int = 10

invariant not_full    { items <= CAPACITY }
invariant not_empty   { items >= 0 }
```

Use this for: buffers, queues, pools, counters.

3.2.2 2. State consistency

```
enum Phase { Idle, Running, Done }
var phase:   Phase
var started: bool
```

```
invariant consistent { phase != Phase.Running || started }
```

"If the machine is running, it must have been started." Use `P != Q || R` or `!P || Q` for implication in invariants.

3.2.3 3. Mutual exclusion

```
var lock_a: bool
var lock_b: bool

invariant mutual_exclusion { !(lock_a && lock_b) }
```

Use this for: locks, tokens, single-owner resources.

3.3 A spec with a bug

```
// bank-buggy.spec

var balance: int
const LIMIT: int = 1000

init { balance = 0 }

action deposit  { balance' = balance + 100 }
action withdraw { balance' = balance - 100 }   // <- no require!

invariant solvent    { balance >= 0 }
invariant within_cap { balance <= LIMIT }
```

```
spectre verify bank-buggy.spec
```

```
Violations:

1.   Invariant violated: solvent
  Counterexample trace (1 step(s)):
    [init]
    [step 1] withdraw: balance: 0 -> -100
    [VIOLATION] invariant "solvent" fails in this state
      balance = -100
```

3.4 Reading a violation trace

```
Violations:

1.   Invariant violated: solvent     <- which invariant failed
  Counterexample trace (1 step(s)):
    [init]                            <- initial state
```

```
    [step 1] withdraw: balance: 0 -> -100   <- which action fired + what changed
    [VIOLATION] invariant "solvent" fails in this state
      balance = -100                  <- concrete value that broke the invariant
```

Every line in the trace shows: - The **step number** - The **action** that fired (or `init`) - The **changed variable** and its before/after values - The **violated invariant** name and the concrete state that broke it

---

3.5 Longer traces

Here's a more interesting bug — withdraw only fails after a freeze/unfreeze cycle:

```
// bank-freeze-bug.spec

enum Status { Active, Frozen }
var balance: int
var status:  Status
const LIMIT: int = 1000

init { balance = 0  status = Status.Active }

action deposit  {
  require status == Status.Active
  require balance + 100 <= LIMIT
  balance' = balance + 100
}

action withdraw {
  require status == Status.Active
  // missing: require balance >= 100
  balance' = balance - 100
}

action freeze   { status' = Status.Frozen  }
action unfreeze { status' = Status.Active  }

invariant solvent { balance >= 0 }
```

```
spectre verify bank-freeze-bug.spec
```

```
Violations:

1.   Invariant violated: solvent
  Counterexample trace (1 step(s)):
    [init]
    [step 1] withdraw: balance: 0 -> -100
    [VIOLATION] invariant "solvent" fails in this state
      balance = -100, status = Active
```

Spectre still finds the shortest path (2 steps), ignoring the freeze/unfreeze actions since they're not needed to trigger the bug.

3.6 Using `--suggest-repairs`

```
spectre verify bank-freeze-bug.spec --suggest-repairs

Repair candidates (CANDIDATE - review before applying):

Violation 1: action 'withdraw' violates 'solvent'
  [1] CANDIDATE strengthen_require: add `require balance >= 100` to action withdraw
      Validation: original violation fixed . no deadlock . reachability unchanged -
SAFE

  [2] CANDIDATE add_invariant_guard: add `when balance >= 100` to action withdraw
      Validation: original violation fixed . deadlock at balance=0 - review required
```

- **SAFE** — the patch fixes the violation and doesn't introduce new problems.
- **review required** — the patch fixes the violation but causes a deadlock or significantly restricts reachability. May be intentional, but check.
- Repairs are **always labelled CANDIDATE** — Spectre never applies them automatically.

Applying repair [1]: add `require balance >= 100` to `withdraw`. Re-verify:

```
spectre verify bank.spec
# Exhaustive verification: traversed 22 states (full reachable space)
```

3.7 Checking multiple invariants

```
invariant solvent     { balance >= 0 }
invariant within_cap  { balance <= LIMIT }
invariant frozen_safe { status != Status.Frozen || balance == 0 }
```

Spectre checks all invariants independently. If multiple are violated, each is reported with its own trace.

3.8 Parameterized safety

```
// producer-consumer.spec
const BUFFER_SIZE: int = 5

var items: int

init { items = 0 }

action produce { require items < BUFFER_SIZE  items' = items + 1 }
action consume { require items > 0            items' = items - 1 }
```

```
invariant buffer_bounded { items <= BUFFER_SIZE }
invariant no_underflow   { items >= 0 }
```

Verify at different sizes by overriding the constant at the command line:

```
spectre verify producer.spec --param BUFFER_SIZE=3
# Exhaustive verification: traversed 4 states (full reachable space)

spectre verify producer.spec --param BUFFER_SIZE=10
# Exhaustive verification: traversed 11 states (full reachable space)
```

Sweep the whole range:

```
spectre verify producer.spec --param-range BUFFER_SIZE=1..20
```

3.9 State space size and `--max-states`

Output line	What it means
`traversed 6 states (full reachable space)`	Full proof — every reachable state checked
`traversed 5000 states (state or depth limit reached)`	Incomplete — violation may still exist beyond the limit

Use `--max-states` to control the limit:

```
spectre verify large.spec --max-states 50000
spectre verify large.spec --max-states -1    # unlimited (may not terminate)
```

Always check whether your result was **exhaustive**. A bounded result is not a proof.

3.10 Invariant-writing checklist

For each resource or constraint in your system, ask:

- **Bounds**: is there a min or max? -> `x >= 0`, `x <= LIMIT`
- **Ordering**: must A happen before B? -> `phase != Phase.B || startedA`
- **Exclusion**: can only one hold at a time? -> `!(P && Q)`
- **Consistency**: do related variables agree? -> `status != Status.Frozen || balance == 0`
- **Monotonicity**: can this only go up/down? -> `count >= prev_count`

3.11 Workflow summary

```
write invariants
      |
```

```
      v
spectre verify
      |
   violation?
   +--+--+
  yes    no
   |      +--> (ok) invariant holds in all reachable states  <- exhaustive result
   v
--suggest-repairs
   |
   v
SAFE candidates  --> apply . re-verify
   |
review-required  --> inspect manually first
```

4 Temporal properties and liveness

Invariants catch bad states. Temporal properties catch bad *trajectories* — things that should eventually happen but don't, or conditions that should hold across a sequence of steps.

4.1 Safety vs liveness

Property kind	Question it answers	Spectre keyword
Safety	"Does anything bad ever happen?"	`invariant`
Liveness	"Does something good eventually happen?"	`temporal`

You need both. A spec where nothing ever happens is trivially safe but useless.

> **BFS and liveness: exploratory only.** The default BFS backend (`spectre verify`) checks reachability and reports `safe` for temporal properties, but this is *not* a valid liveness proof. BFS cannot distinguish "eventually holds on all fair executions" from "reachable but not guaranteed." If your spec contains `eventually`, `until`, `P -> Q` (leads-to), `WF`, or `SF`, use `--backend tlc` for a sound result. Running BFS on a spec with liveness properties gives you a quick sanity check — not a guarantee. If you want an explicit error rather than a silent exploratory result, run `spectre verify --backend tlc`.

4.2 `always` — a condition that holds forever

```
temporal always_positive { always(balance >= 0) }
```

This is equivalent to an invariant — but expressed in the temporal language. Use `invariant` for simple per-state checks; use `always` when you're building up compound temporal expressions.

Full example — traffic light that cycles through valid states:

```
enum Light { Red, Yellow, Green }
var signal: Light

init { signal = Light.Red }

action to_green  { require signal == Light.Red     signal' = Light.Green  }
action to_yellow { require signal == Light.Green   signal' = Light.Yellow }
action to_red    { require signal == Light.Yellow  signal' = Light.Red    }

invariant always_lit { signal == Light.Red || signal == Light.Yellow || signal == Light.Green }
temporal  never_off  { always(signal == Light.Red || signal == Light.Yellow || signal == Light
```

```
spectre verify traffic.spec
# Exhaustive verification: traversed 3 states (full reachable space)
# Found no violations.
```

4.3 `eventually` — something must happen

```
temporal reaches_max { eventually(value == MAX) }
```

This says: on every execution path, `value` must at some point equal `MAX`.

Counter that counts to MAX and stops:

```
var value: int
const MAX: int = 4

init { value = 0 }

action increment { require value < MAX  value' = value + 1 }

invariant non_negative { value >= 0 }
temporal  reaches_max  { eventually(value == MAX) }
```

```
spectre verify counter.spec
# Exhaustive verification: traversed 5 states (full reachable space)
# Found no violations.
```

Now add a `reset` action:

```
action reset { value' = 0 }
```

```
spectre verify counter.spec
# Exhaustive verification: traversed 5 states (full reachable space)
# Found no violations.
```

BFS finds no violation because `value == MAX` is still reachable (increment four times). BFS is the **learning backend** — it checks whether a property *can* become true in the reachable state space, not whether it is *guaranteed* to become true on every execution. Because `reset` can fire repeatedly and deny increment, the property can fail on an unfair execution. To detect that, add `WF(increment)` and validate with `--backend tlc`:

```
spectre verify counter.spec --backend tlc
# TLC: VIOLATION - reaches_max not satisfied under a fair scheduler
# (reset can fire indefinitely, starving increment)
```

See the Fairness chapter for WF/SF and the TLA+ and Apalache chapter for the TLC backend.

4.4 `until` — hold one condition until another

```
temporal safe_until_done { (errors == 0) until (phase == Phase.Done) }
```

This says: `errors == 0` must hold on every step *until* `phase == Phase.Done` is reached.

Download manager example:

```
enum Phase { Downloading, Done }
var phase: Phase
var error: bool

init { phase = Phase.Downloading  error = false }

action download { require phase == Phase.Downloading  phase' = Phase.Done }
action fail     { require phase == Phase.Downloading  error' = true       }

temporal no_errors_until_done { (!error) until (phase == Phase.Done) }
```

```
spectre verify download.spec
# Exhaustive verification: traversed 4 states (full reachable space)
# Violations:
# 1. Temporal Property (no_errors_until_done): P doesn't hold before Q
#    Counterexample: fail
```

`fail` fires before `download`, setting `error=true` while `phase` is still `Downloading`. Fix: add `require !error` to `fail`, or remove the `fail` action if errors are not allowed before completion.

4.5 Leads-to — if P then eventually Q

```
temporal request_served { always(pending -> eventually(!pending)) }
```

Every time a request becomes pending, it must eventually be cleared. This is the "leads-to" pattern: `always(P -> eventually(Q))`.

Spectre uses LTL (Linear Temporal Logic) semantics over execution traces. `always(P -> eventually(Q))` means: in every suffix of every execution where P holds, Q eventually becomes true. This is equivalent to the leads-to operator P ~> Q. BFS explores traces and checks reachability, which is not sufficient to validate leads-to — use `--backend tlc` or `--backend apalache` for this pattern.

```
spectre verify server.spec --backend tlc
# -> passes if serve is reachable from every state where pending=true
```

The correct way to write and verify a leads-to property:

```
var pending: bool
var served:  bool

init { pending = false  served = false }

action request { require !pending  pending' = true               }
action serve   { require pending   pending' = false  served' = true }

temporal request_served { always(pending -> eventually(!pending)) }
temporal wf_serve       { WF(serve) }
```

```
spectre verify server.spec --backend tlc
# TLC: verified - pending always leads to !pending (WF ensures serve fires)
```

See the TLA+ and Apalache chapter for the TLC backend details.

4.6 Combining temporal properties

A spec can have multiple temporal properties. Spectre checks all of them.

```
var value:    int
var response: bool
const MAX: int = 5

init { value = 0  response = false }

action increment { require value < MAX  value' = value + 1 }
action respond   { response' = true                         }

temporal always_bounded { always(value <= MAX)         }
temporal reaches_max    { eventually(value == MAX)     }
temporal gets_response  { eventually(response)         }
```

What each property covers:

```
States:  *---*---*---*---*---*
        v=0 v=1 v=2 v=3 v=4 v=5=MAX

always_bounded:  every node  ->  value <= MAX       (ok) per-state
reaches_max:     some path   ->  reaches value=MAX  (ok) across paths
gets_response:   every run   ->  response fires     (ok) reachability
```

For `always(P -> eventually(Q))` (leads-to), use `--backend tlc` — BFS trace exploration does not validate it correctly.

4.7 Temporal verification with large state spaces

If BFS hits `--max-states`, temporal properties are only checked over explored states:

```
spectre verify large.spec --max-states 5000
# bounded -- hit state limit
# temporal properties checked over 5000 states only
# Use --backend tlc for exhaustive temporal verification
```

Use `--backend tlc` or `--backend apalache` for exhaustive temporal verification on large specs (see the TLA+ and Apalache chapter).

4.8 Quick-reference

Operator	Meaning	Use when
`always(P)`	P holds in every reachable state	Global safety, expressed temporally
`eventually(P)`	Every path eventually reaches P	Progress / reachability goals
`P until Q`	P holds on all steps until Q	Precondition that must hold until a goal
`always(P -> eventually(Q))`	Every P event leads to Q	Request/response, lock/unlock pairs — use `--backend tlc`
`WF(action)`	Action fires if continuously enabled	Fairness chapter
`SF(action)`	Action fires if infinitely often enabled	Fairness chapter

5 Fairness — WF and SF

Temporal properties say *what* must eventually happen. Fairness conditions say *which executions count* when checking those properties. Without fairness, the verifier is allowed to assume a scheduler that permanently ignores actions — which makes most liveness properties unprovable.

5.1 Why fairness matters

```
var token: bool

init { token = false }

action grant  { require !token  token' = true  }
action revoke { require token   token' = false }

temporal eventually_granted { eventually(token) }
```

```
spectre verify token.spec
# Exhaustive verification: traversed 2 states (full reachable space)
# Found no violations.
```

BFS finds no violation because from the initial state (`token = false`), only `grant` is enabled — so `token` is forced to become true. BFS checks reachability, and `token = true` is reachable.

The *fairness* issue would appear on a spec where two actions compete. In the mutex below, without `WF(p1_acquire)` the scheduler could run p2 forever and starve p1. Declare `WF(action)` to document the fairness intent and validate it with `--backend tlc`.

```
Without WF (token spec):   token starts false -> only grant fires -> no issue
With WF(grant):            documents that grant must fire if continuously enabled
```

5.2 Weak fairness: WF(action)

> "If an action stays enabled, it must eventually fire."

```
temporal wf_grant { WF(grant) }
```

Add this to the spec:

```
var token: bool

init { token = false }

action grant  { require !token  token' = true  }
action revoke { require token   token' = false }

temporal eventually_granted { eventually(token)  }
temporal wf_grant           { WF(grant)          }
```

```
spectre verify token.spec
# Exhaustive verification: traversed 2 states (full reachable space)
# Found no violations.
```

WF rules out executions where `grant` is permanently skipped while enabled.

5.2.1 Mutex with progress guarantee

```
enum ProcState { Waiting, Running, Done }
var p1:   ProcState
var p2:   ProcState
var lock: bool

init { p1 = ProcState.Waiting  p2 = ProcState.Waiting  lock = false }

action p1_acquire { require p1 == ProcState.Waiting && !lock  p1' = ProcState.Running  lock' =
action p1_release { require p1 == ProcState.Running           p1' = ProcState.Done     lock' =
action p2_acquire { require p2 == ProcState.Waiting && !lock  p2' = ProcState.Running  lock' =
action p2_release { require p2 == ProcState.Running           p2' = ProcState.Done     lock' =
```

```
invariant mutex      { !(p1 == ProcState.Running && p2 == ProcState.Running) }
temporal p1_progress { WF(p1_acquire) }
temporal p2_progress { WF(p2_acquire) }
temporal p1_finishes { eventually(p1 == ProcState.Done) }
temporal p2_finishes { eventually(p2 == ProcState.Done) }
```

```
spectre verify mutex.spec
# Exhaustive verification: traversed 8 states (full reachable space)
# Found no violations.
```

Without `WF(p1_acquire)`, `p1_finishes` would fail — the scheduler could run p2 forever and starve p1.

5.3 Strong fairness: SF(action)

> "If an action is enabled infinitely often, it must fire infinitely often."

The difference from WF: SF handles actions that are *intermittently* enabled.

```
var connected: bool
var retries:   int

init { connected = false  retries = 0 }

action disconnect { require connected   connected' = false           }
action connect    { require !connected  connected' = true            }
action retry      { require !connected  retries'   = retries + 1     }

temporal sf_retry { SF(retry) }
temporal eventually_connected { eventually(connected) }
```

`retry` is only enabled when `connected = false` — which isn't always the case. WF(retry) wouldn't help here because `retry` isn't *continuously* enabled. SF(retry) says: whenever `retry` becomes enabled again, it must fire.

5.3.1 WF vs SF at a glance

	WF(A)	SF(A)
Fires when	Continuously enabled	Infinitely often enabled
Strength	Weaker	Stronger
Use for	Lock acquisition, queue dequeue	Retry, intermittent tasks
Example	Mutex `acquire`	Network `retry`

5.4 WF and SF in the same spec

You can mix them freely:

```
temporal acquire_wf   { WF(acquire)  }   // always-enabled action
temporal retry_sf     { SF(retry)    }   // intermittently-enabled action
temporal done_for_all { eventually(p1 == ProcState.Done && p2 == ProcState.Done) }
```

Spectre checks all temporal properties together.

5.5 BFS for learning, TLC for validation

BFS is the learning backend. It explores the reachable trace space and reports cases where an enabled action is never taken — useful for developing intuition about your spec's liveness behaviour. For validated fairness results, use the TLC backend:

```
spectre verify mutex.spec --backend tlc
```

TLC performs proper SCC-based (strongly connected component) fairness checking — the same algorithm validated in production TLA+ tooling for two decades. It translates `WF(action)` to `WF_vars(Action)` in TLA+ and lets TLC's cycle-detection engine do the rest. BFS is great for rapid iteration; TLC gives the authoritative answer.

```
WF/SF in Spectre
       |
       v
spectre verify  (BFS - learning mode, trace exploration)
       |
  confident? --yes--> done
       |
       no (need validated fairness, complex interactions, large spec)
       |
       v
spectre verify --backend tlc
(SCC-based, validated liveness)
```

See the TLA+ and Apalache chapter for full TLC and Apalache backend details.

5.6 Fairness checklist

When writing a spec with liveness properties:

- Add `WF(action)` for every action that represents a "service obligation" (lock acquisition, queue drain, message delivery)
- Add `SF(action)` for actions that are intermittently available (retries, reconnects, elections)
- If `eventually(P)` fails unexpectedly, add fairness before debugging the spec logic
- For critical liveness proofs, validate with `--backend tlc`

6 Concurrent and distributed systems

Spectre models concurrency by letting any action fire at any time. BFS explores every possible interleaving. This makes it easy to find races that are nearly impossible to trigger in testing.

6.1 What's different about concurrent specs

Three things you don't see in sequential specs:

- **Shared state** — multiple actors read and write the same variables
- **Interleaving** — actions from different actors can fire in any order
- **Races** — two actors attempt incompatible transitions simultaneously

You don't need threads or processes. You model each actor as a set of actions and let Spectre find the bad interleavings.

6.2 Start with a broken mutex

```
// mutex-broken.spec
var p1_in_cs: bool
var p2_in_cs: bool

init { p1_in_cs = false  p2_in_cs = false }

action p1_enter { p1_in_cs' = true  }
action p1_exit  { p1_in_cs' = false }
action p2_enter { p2_in_cs' = true  }
action p2_exit  { p2_in_cs' = false }

invariant mutex { !(p1_in_cs && p2_in_cs) }
```

```
spectre verify mutex-broken.spec
# Violations:
# 1. Invariant violated: mutex
#    Action 'p1_enter' would violate invariants: [invariant mutex violated]
#    Counterexample trace (1 step(s)):
#      [init]
#      [step 1] p2_enter: p2_in_cs: false -> true
#      [VIOLATION] invariant "mutex" fails in this state
```

Spectre found the race in 2 steps.

6.3 Fix it with a lock

```
// mutex.spec
var p1_in_cs: bool
```

```
var p2_in_cs: bool
var lock:     bool

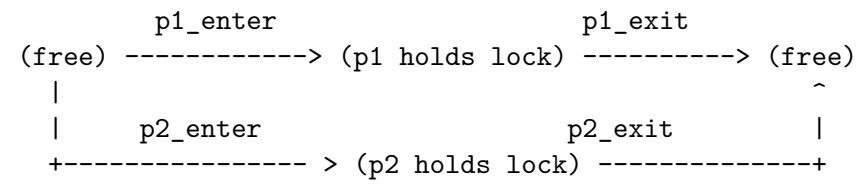
init { p1_in_cs = false  p2_in_cs = false  lock = false }

action p1_enter { require !lock    p1_in_cs' = true   lock' = true  }
action p1_exit  { require p1_in_cs  p1_in_cs' = false  lock' = false }
action p2_enter { require !lock    p2_in_cs' = true   lock' = true  }
action p2_exit  { require p2_in_cs  p2_in_cs' = false  lock' = false }

invariant mutex      { !(p1_in_cs && p2_in_cs) }
invariant lock_owner { !lock || p1_in_cs || p2_in_cs }
temporal  p1_progress { WF(p1_enter) }
temporal  p2_progress { WF(p2_enter) }
```

```
spectre verify mutex.spec
# Exhaustive verification: traversed 3 states (full reachable space)
# Found no violations.
```

State machine for this model:



Only one process can hold the lock — `p2_enter` and `p1_enter` both `require !lock`, so they can never both fire from the same state.

6.4 Producer-consumer with a bounded buffer

```
const CAPACITY: int = 3
var items: int

init { items = 0 }

action produce { require items < CAPACITY  items' = items + 1 }
action consume { require items > 0         items' = items - 1 }

invariant no_overflow  { items <= CAPACITY }
invariant no_underflow { items >= 0        }
temporal  eventually_full { eventually(items == CAPACITY) }
```

```
spectre verify buffer.spec
# Exhaustive verification: traversed 4 states (full reachable space)
# Found no violations.
```

```
spectre verify buffer.spec --param CAPACITY=10
# Exhaustive verification: traversed 11 states (full reachable space)
```

Adding a second producer (`produce2`) immediately doubles the state space — BFS handles it automatically.

6.5 Distributed message queue

Three-role system: producers enqueue, consumers dequeue, an accounting invariant keeps them honest.

```
const MAX_QUEUE: int = 5
var queue_len: int
var produced:  int
var consumed:  int

init { queue_len = 0  produced = 0  consumed = 0 }

action enqueue {
  require queue_len < MAX_QUEUE
  require produced < MAX_QUEUE
  queue_len' = queue_len + 1
  produced'  = produced + 1
}

action dequeue {
  require queue_len > 0
  queue_len' = queue_len - 1
  consumed'  = consumed + 1
}

invariant queue_bounded  { queue_len <= MAX_QUEUE }
invariant queue_positive { queue_len >= 0         }
invariant accounting     { produced == consumed + queue_len }
```

```
spectre verify queue.spec
# Exhaustive verification: traversed 21 states (full reachable space)
# Found no violations.
```

The `accounting` invariant is the most valuable one here. It catches any race where messages are created or destroyed without both counters updating — a class of bugs that integration tests rarely find.

To add a liveness property (queue must eventually drain), use `--backend tlc` since `always(P -> eventually(Q))` requires SCC-based checking:

```
spectre verify queue.spec --backend tlc
# -> verifies always(queue_len > 0 -> eventually(queue_len == 0)) with WF(dequeue)
```

6.6 Good invariants for concurrent specs

Pattern	Invariant example
Mutual exclusion	`!(p1_in_cs && p2_in_cs)`
Accounting identity	`produced == consumed + in_flight`
State consistency	`!locked || owner_known`
Bounded resource	`active_connections <= MAX_CONN`
No phantom state	`!waiting || (request_sent && !response_received)`

Write at least one invariant of each type that applies to your system.

6.7 When the state space gets large

Three-process or three-role specs can hit tens of thousands of states:

```
spectre verify 3-party.spec --max-states 50000
# bounded -- hit state limit (50000 states)
```

Options when BFS doesn't finish:

Option	When to use
`--max-states 200000`	More time, same algorithm
`--backend apalache`	Symbolic — handles larger spaces
`--por`	Partial-order reduction — skips equivalent interleavings
Split the spec	Verify each component separately first

```
spectre verify 3-party.spec --backend apalache
# uses Apalache's symbolic engine - see the TLA+ and Apalache chapter
```

6.8 Concurrency workflow

```
Identify shared variables
         |
         v
Write one action per actor per operation
         |
         v
Add require guards for preconditions
         |
         v
Write mutex + accounting invariants
         |
         v
spectre verify  (BFS - learning mode)
```

```
        |
    violation? --yes--> read the counterexample trace
        |                       |
        no                      v
        |              strengthen requires or add lock variable
        v
Add WF/SF temporal properties
        |
        v
spectre verify --backend tlc  (for validated fairness)
```

7 Modules, parameters, and reuse

Constants let you change a spec's values without editing the logic. Modules let you split a large spec into named, reusable pieces.

7.1 Constants — changing the spec without editing it

```
const CAPACITY:  int = 10
const THRESHOLD: int = 7
var items: int

invariant not_full       { items <= CAPACITY  }
invariant below_threshold { items <  THRESHOLD }
```

Three ways to set constant values:

- **Default** — just run `spectre verify queue.spec`
- **Override** — `spectre verify queue.spec --param CAPACITY=20`
- **Sweep** — `spectre verify queue.spec --param-range CAPACITY=1..20`

7.1.1 Parameter sweep

```
spectre verify queue.spec --param-range CAPACITY=1..10
```

```
CAPACITY  States  Result
--------  ------  ------
       1       3  safe (exhaustive)
       2       6  safe (exhaustive)
       3      10  safe (exhaustive)
       4      15  safe (exhaustive)
       5      21  safe (exhaustive)
      10      66  safe (exhaustive)
```

Sweeping is useful for finding the exact value where a spec breaks or where the state space starts to explode.

7.2 Modules — reusable spec components

A module bundles variables, constants, init, actions, and invariants under a name.

```
module Counter {
  var count: int
  const MAX: int = 10

  init { count = 0 }

  action increment { require count < MAX  count' = count + 1 }
  action decrement { require count > 0    count' = count - 1 }

  invariant non_negative  { count >= 0   }
  invariant within_bounds { count <= MAX }
}
```

Import and compose it in another spec:

```
import Counter

var active: bool

init { active = true }

action toggle { active' = !active }

invariant active_non_negative { !active || Counter.count >= 0 }
```

```
spectre verify composed.spec
# Exhaustive verification: traversed 22 states (full reachable space)
```

7.3 Module inheritance — parameterized reuse

```
module Semaphore {
  const SLOTS: int = 1
  var held: int

  init { held = 0 }

  action acquire { require held < SLOTS  held' = held + 1 }
  action release { require held > 0      held' = held - 1 }

  invariant valid { held >= 0 && held <= SLOTS }
}

module Mutex  extends Semaphore { const SLOTS = 1 }
module RWLock extends Semaphore { const SLOTS = 3 }
```

How `extends` works: Spectre modules use **parameterized macro substitution**, not OOP inheritance. `extends` copies all declarations from the parent module into the child scope and substitutes overridden constants. There is no dynamic dispatch, method override, or multiple independent instances of the same module. It is equivalent to textual include with constant replacement — a convenience for sharing invariants and structure, not a class hierarchy.

The same `valid` invariant covers both variants — you write it once and get it for free in every subtype.

```
spectre verify mutex.spec   # SLOTS=1 -> 3 states
spectre verify rwlock.spec  # SLOTS=3 -> 5 states
```

7.4 Splitting a large spec into modules

Before — one flat file, hard to navigate:

```
var user_count:    int
var session_count: int
var db_connections: int
const MAX_USERS:    int = 1000
const MAX_SESSIONS: int = 5000
const MAX_CONNS:    int = 20

action add_user    { ... }
action remove_user { ... }
action open_session { ... }
action close_session { ... }
action open_connection { ... }
action close_connection { ... }
// ... 40 more lines
```

After — three focused modules:

```
// users.spec
module UserManager {
  var user_count: int
  const MAX_USERS: int = 1000

  init { user_count = 0 }

  action add_user    { require user_count < MAX_USERS  user_count' = user_count + 1 }
  action remove_user { require user_count > 0          user_count' = user_count - 1 }

  invariant valid_count { user_count >= 0 && user_count <= MAX_USERS }
}

// sessions.spec
module SessionManager {
```

```
  var session_count: int
  const MAX_SESSIONS: int = 5000

  init { session_count = 0 }

  action open_session  { require session_count < MAX_SESSIONS  session_count' = session_count +
  action close_session { require session_count > 0             session_count' = session_count -

  invariant valid_sessions { session_count >= 0 && session_count <= MAX_SESSIONS }
}

// app.spec - the composition
import UserManager
import SessionManager

invariant sessions_per_user {
  SessionManager.session_count <= UserManager.user_count * 10
}
```

Benefits of splitting: - Each module verifies independently — catch bugs early - Modules are reusable across projects - Invariants at the composition layer catch cross-module bugs - State spaces stay small per module

---

7.5 Workflow: from flat to modular

```
Single flat spec
      |
      v
Identify natural groupings
  (resources, actors, subsystems)
      |
      v
Extract each group into a module
      |
      v
spectre verify each module independently
      |
      v
Compose with import
      |
      v
Add cross-module invariants
      |
      v
spectre verify composition
```

7.6 Constant best practices

- Always set a **sensible default** — specs should verify out of the box
- Use **ALL_CAPS** names for constants (`MAX`, `CAPACITY`, `TIMEOUT`) to distinguish from variables
- Keep constants **small for development** — verify with `--param N=3` before `--param N=100`
- Sweep constant values in CI to catch spec regressions across the whole value range

8 Collections and distributed specifications

Scalar variables (`int`, `bool`, enum) are enough for counters and state machines. Real distributed protocols need something more: sets of nodes, vote tallies, message queues. Spectre has first-class `Set<T>`, `Map<K,V>`, and `List<T>` types for exactly this.

8.1 The three collection types

Type	Use for	Example init
`Set<T>`	Unordered membership: nodes, voters, committed replicas	`nodes = {1, 2, 3}`
`Map<K,V>`	Key->value associations: votes, roles, config	`votes = {}`
`List<T>`	Ordered sequences: message queues, logs, histories	`queue = []`

Declare them exactly like any variable:

```
var members:  Set<int>
var role:     Map<int, bool>
var log:      List<int>
```

8.2 Set operations

```
var S: Set<int>
var T: Set<int>

init { S = {1, 2, 3}  T = {3, 4} }

action grow   { S' = S.union({4}) }
action shrink { S' = S.difference({1}) }
```

The full method set:

Method	Returns	Meaning
`S.union(T)`	`Set<T>`	All elements of S or T
`S.intersection(T)`	`Set<T>`	Elements in both S and T
`S.difference(T)`	`Set<T>`	Elements in S but not T
`S.contains(x)`	`bool`	True if x is in S
`x in S`	`bool`	Same — infix form, reads naturally in guards
`S.size()`	`int`	Number of elements
`S.filter(fn)`	`Set<T>`	Elements matching predicate
`S.toList()`	`List<T>`	Convert to list (order unspecified)

The `in` operator is the most natural way to write membership guards:

```
action vote {
  require !(node_id in voted)   // has not voted yet
  voted' = voted.union({node_id})
}
```

8.3 Map operations

```
var scores: Map<int, int>
init { scores = {} }

action record { scores' = scores.put(42, 100) }
```

Method	Returns	Meaning
`M.put(k, v)`	`Map<K,V>`	Returns map with key k set to v
`M.get(k)`	`V`	Value for key k (error if missing)
`k in M`	`bool`	True if k has an entry in M
`M.contains(k)`	`bool`	Same — method form
`M.size()`	`int`	Number of entries

8.4 List operations

```
var queue: List<int>
init { queue = [] }

action enqueue { queue' = queue.append(42) }
```

Method	Returns	Meaning
`L.append(x)`	`List<T>`	List with x added at end
`L.head()`	`T`	First element (error if empty)
`L.tail()`	`List<T>`	List without first element

Method	Returns	Meaning
`L.size()`	`int`	Number of elements
`L.filter(fn)`	`List<T>`	Elements matching predicate
`L.toSet()`	`Set<T>`	Convert to set (drops order/duplicates)

8.5 Example: Two-Phase Commit

Two-Phase Commit (2PC) is a classic distributed transaction protocol. It requires sets to reason about: which nodes have voted, which have committed, and whether the results are consistent.

```
Coordinator                    Nodes
     |                           |
     |---- Prepare ------------->|
     |                           |
     |<--- Vote (yes/no) --------|  each node votes independently
     |                           |
  all yes?                       |
     |---- Commit/Abort -------->|
     |                           |
     |<--- Ack ------------------|
```

The spec (`examples/two_phase_commit.spec`):

```
const NUM_NODES: int = 3

enum CoordPhase { Preparing, Committed, Aborted }

var coord:     CoordPhase
var voted_yes: Set<int>   // nodes that voted yes
var voted_no:  Set<int>   // nodes that voted no
var committed: Set<int>   // nodes that applied commit
var aborted:   Set<int>   // nodes that applied abort

init {
  coord     = CoordPhase.Preparing
  voted_yes = {}
  voted_no  = {}
  committed = {}
  aborted   = {}
}

action node1_vote_yes {
  require coord == CoordPhase.Preparing
  require !(1 in voted_yes) && !(1 in voted_no)
  voted_yes' = voted_yes.union({1})
}
```

```
// ... node2_vote_yes, node3_vote_yes, node1_vote_no, ... (same pattern)

action coordinator_commit {
  require coord == CoordPhase.Preparing
  require voted_yes.size() == NUM_NODES
  require voted_no.size() == 0
  coord' = CoordPhase.Committed
}

action coordinator_abort {
  require coord == CoordPhase.Preparing
  require voted_no.size() > 0
  coord' = CoordPhase.Aborted
}

action node1_commit {
  require coord == CoordPhase.Committed
  require !(1 in committed)
  committed' = committed.union({1})
}

// ... node2_commit, node3_commit, node1_abort, ... (same pattern)
```

The safety invariants:

```
// A node never both commits and aborts (atomicity)
invariant atomicity {
  committed.intersection(aborted).size() == 0
}

// A node only commits if the coordinator committed
invariant commit_requires_coord_committed {
  committed.size() == 0 || coord == CoordPhase.Committed
}

// No node votes both yes and no
invariant votes_disjoint {
  voted_yes.intersection(voted_no).size() == 0
}

// The coordinator only commits on unanimous yes vote
invariant unanimous_commit {
  coord != CoordPhase.Committed || voted_yes.size() == NUM_NODES
}
```

Run it:

```
spectre verify examples/two_phase_commit.spec
# Exhaustive verification: traversed 673 states (full reachable space)
# Found no violations.
```

673 states covers every possible interleaving of votes and decisions across all 3 nodes. BFS finds this exhaustively in milliseconds — a great use of the learning backend before scaling up.

8.6 Why collection specs find bugs scalars can't

Consider a buggy 2PC coordinator that commits as soon as *any* node votes yes (not all):

```
action coordinator_commit_buggy {
  require coord == CoordPhase.Preparing
  require voted_yes.size() > 0    // BUG: should be == NUM_NODES
  coord' = CoordPhase.Committed
}
```

The `unanimous_commit` invariant immediately catches this:

```
Violation 1: invariant "unanimous_commit"
  Step 0: [coord=Preparing, voted_yes={}, voted_no={}, ...]
  Step 1: [coord=Preparing, voted_yes={1}, voted_no={},  ...]  <- node 1 votes yes
  Step 2: [coord=Committed, voted_yes={1}, voted_no={},  ...]  <- coordinator commits too early
  violated: coord == Committed but voted_yes.size() == 1 != 3
```

This class of bug is invisible in scalar-variable specs. The set tracks *which* nodes voted, not just *how many* — making it possible to state the unanimous condition precisely.

8.7 State-space note: collections multiply states

Each unique set value is a distinct state. `Set<int>` over `{1,2,3}` has 8 possible values (every subset). With 5 variables like the 2PC spec, the product is large — but BFS still exhausts it at 673 states because the invariants and guards constrain which subsets are reachable.

For larger protocols (Raft, Paxos, 5+ nodes), state spaces quickly exceed BFS capacity. That's when you escalate:

```
Small protocol (2-3 nodes)   -->  --backend bfs        (learning, exhaustive)
Medium protocol (4-5 nodes)  -->  --backend tlc        (exhaustive, validated)
Large protocol (6+ nodes)    -->  --backend apalache   (symbolic, exhaustive)
Liveness / leader election   -->  --backend tlc        (validated fairness)
```

A note on Z3 and collections: The Z3 backend (bounded model checking) translates `Set<T>` to SMT-LIB2 sets and can catch shallow set bugs. However, `Cardinality` checks over `Set<int>` are unreliable in Z3 because integer sets are infinite — Z3 may report spurious violations. Use Z3 for set membership and boolean checks; use TLC or Apalache for specs with `.size()` comparisons on integer sets.

8.8 Collection syntax quick-reference

```
// Set literal
var S: Set<int>
```

```
init { S = {1, 2, 3} }

// Empty set / empty map / empty list
var empty_set:  Set<int>    -- init to {}
var empty_map:  Map<int,bool>  -- init to {}
var empty_list: List<int>   -- init to []

// Membership test (in a require or invariant)
require x in S
invariant inv { !(x in S) || x > 0 }

// Set update (all assignments return new values - no mutation)
S' = S.union({x})
S' = S.difference({x})

// Map update
M' = M.put(key, value)

// List update
L' = L.append(x)
let head = L.head()   // read first element (use in expressions/requires)
let rest = L.tail()   // read tail (use in expressions/requires)
```

9 Reading counterexamples and debugging

When verification fails, Spectre gives you the shortest possible path to the violation. Here's how to read it and fix the spec.

9.1 Anatomy of a violation trace

```
Violations:

1.   Invariant violated: solvent              <- which invariant failed
  Counterexample trace (4 step(s)):           <- BFS shortest path
    [init]                                    <- starting state
    [step 1] deposit:  balance: 0 -> 100       <- action + what changed
    [step 2] deposit:  balance: 100 -> 200
    [step 3] withdraw: balance: 200 -> 200     <- no change (wrong action)
    [step 4] withdraw: balance: 200 -> -100
    [VIOLATION] invariant "solvent" fails in this state
      balance = -100                          <- concrete value that broke it
      status = Active
```

Every part of the output tells you something:

Part	What it tells you
Invariant violated	Which invariant failed
Counterexample (N step(s))	BFS found the *shortest* path — no shorter path exists
Each step	Which action ran + which variable changed and how
[VIOLATION]	The exact state where the invariant first fails
Variable list	The full state that broke the invariant

9.2 Five-step debugging workflow

```
1. Read the violated invariant
         |  "solvent: balance >= 0"
         v
2. Look at the violation step
         |  "withdraw: balance = -100"
         v
3. Read backwards - what enabled this action?
         |  Previous state: balance = 200, no require check
         v
4. Find the missing guard
         |  withdraw has no "require balance >= amount"
         v
5. Add require, re-verify
         |  spectre verify bank.spec  ->  (ok) safe
         v
       done
```

9.3 Worked example: the transfer bug

```
// transfer-broken.spec
var alice: int
var bob:   int

init { alice = 200  bob = 100 }

action transfer {
  // BUG: no check that alice has enough
  alice' = alice - 100
  bob'   = bob   + 100
}

invariant alice_solvent { alice >= 0       }
invariant bob_solvent   { bob   >= 0       }
invariant total_fixed   { alice + bob == 300 }
```

```
spectre verify transfer-broken.spec

Violations:

1.   Invariant violated: alice_solvent
  Action 'transfer' would violate invariants: [invariant alice_solvent violated]
  Counterexample trace (2 step(s)):
    [init]
    [step 1] transfer: alice: 200 -> 100, bob: 100 -> 200
    [step 2] transfer: alice: 100 -> 0,   bob: 200 -> 300
    [VIOLATION] invariant "alice_solvent" fails in this state
      alice = 0
      bob = 300
```

Spectre reports the state where the *next* `transfer` would violate `alice_solvent` (alice=0 -> -100). The fix — add a require:

```
action transfer {
  require alice >= 100
  alice' = alice - 100
  bob'   = bob   + 100
}
```

```
spectre verify transfer-fixed.spec
# Exhaustive verification: traversed 3 states (full reachable space)
# Found no violations.
```

Notice `total_fixed` — it catches bugs where money is created or destroyed. You get that protection for free just by writing the accounting identity as an invariant.

9.4 Using `--suggest-repairs`

When a violation is found, `--suggest-repairs` analyses the action and invariant and proposes patches.

```
spectre verify transfer-broken.spec --suggest-repairs

Repair candidates (CANDIDATE - review before applying):

Violation 1: action 'transfer' violates 'alice_solvent'
  [1] CANDIDATE strengthen_require: add `require alice >= 100` to action transfer
      Validation: original violation fixed . no deadlock . reachability unchanged -
SAFE
  [2] CANDIDATE add_invariant_guard: add `when alice >= 100` to action transfer
      Validation: original violation fixed . deadlock at alice=0 - review required
```

Two candidate types — pick the right one for your situation:

Candidate type	Pick when	Risk
`strengthen_require`	The action simply shouldn't fire without the condition	Minimal — action is disabled, not blocked
`add_invariant_guard`	You want the action blocked at the invariant boundary	Can deadlock if the state has no other exit

9.4.1 Understanding deadlock warnings

`Validation: original violation fixed . deadlock at alice=0 - review required`

A deadlock is a reachable state where **no action is enabled**. This isn't always wrong — sometimes "nothing can happen" is correct behavior. Check:

- Is that final state intentional? (e.g., a `Done` phase)
- Or is the system permanently stuck? (a real bug in the repair)

9.5 Multiple violations

Spectre checks all invariants and reports the first violation it finds (shortest BFS path). If you have multiple bugs, fix them one at a time — each fix may expose the next shortest path.

```
spectre verify buggy.spec
# (x) violation -- invariant 'alice_solvent' ...

# Fix alice_solvent, then:
spectre verify partially-fixed.spec
# (x) violation -- invariant 'total_fixed' ...
```

9.6 Dealing with bounded results

`bounded -- hit state limit (5000 states)`

This means BFS stopped before exploring everything. Your spec might be safe — or the violation might be deeper than the limit. Your options:

1. `--max-states 50000` — increase the limit and try again
2. `--max-states -1` — unlimited (may not terminate for large specs)
3. `--backend apalache` — symbolic verification, handles large spaces exhaustively
4. `--backend z3 --bmc-depth 30` — bounded check to depth 30; fast, not exhaustive

```
spectre verify large.spec --max-states 50000
spectre verify large.spec --backend apalache
spectre verify large.spec --backend z3 --bmc-depth 30
```

Always check whether the output says `exhaustive` or `bounded` — it matters for how much you can trust the result.

9.7 Common patterns and their fixes

Symptom	Likely cause	Fix
Negative integer variable	Missing lower-bound require	`require x >= 0` before decrement
Variable exceeds param	Missing upper-bound require	`require x < MAX` before increment
Enum in wrong state	Action enabled in wrong phase	`require phase == Phase.Expected`
Two actors in critical section	No mutual exclusion	Add lock variable + requires
Accounting identity broken	Value leaked or doubled	Check all actions modify both sides

9.8 Tips for writing debuggable specs

- **Write accounting identities** — `alice + bob == TOTAL` catches money bugs for free
- **Name invariants clearly** — the name is the first thing you see in a violation
- **One concept per action** — large multi-step actions are hard to trace
- **Start with small params** — `--param N=3` first, then scale up
- **Verify after every change** — short iteration loops catch bugs while the context is fresh

10 Choosing a verification backend

`spectre verify` supports four backends. BFS is the learning backend — zero setup, instant traces, great while you are writing and exploring a spec. The others are production backends for when you need exhaustive or validated results.

10.1 Two tracks: learning vs. production

```
Learning track
  +- --backend bfs  (default)
       No setup. Instant traces. Great for writing specs and debugging.
       Liveness results are exploratory - use TLC to validate them.

Production track
  +- --backend tlc        Validated liveness + fairness (WF/SF)
  +- --backend apalache   Exhaustive safety on large state spaces
```

```
  +- --backend z3         Fast shallow bug hunting in CI
```

Pick BFS while you are learning or iterating. Graduate to a production backend before sign-off.

10.2 The four backends at a glance

```
spectre verify myspec.spec
          |
          |  --backend bfs        (default, learning)
          |  --backend tlc
          |  --backend apalache
          |  --backend z3  --bmc-depth N
          |
          v
    +-----------+--------------+-------------+---------------+
    |   BFS     |    TLC       |  Apalache   |     Z3        |
    +-----------+--------------+-------------+---------------+
    | Learning  | Exhaustive   | Exhaustive  | Bounded       |
    | & explore | + validated  | symbolic    | up to depth N |
    |           | fairness     | large spaces |               |
    +-----------+--------------+-------------+---------------+
    | No deps   | TLA+ tools   | Apalache bin | z3 binary     |
    |           | (jar or bin) |              |               |
    +-----------+--------------+-------------+---------------+
```

10.3 Quick decision guide

```
Writing a new spec or debugging?
  +-- yes --> --backend bfs  (default, zero setup, instant traces)

Need validated liveness or WF/SF fairness?
  +-- yes --> --backend tlc  (exact SCC-based cycle detection)

State space too large for BFS (> 100k states)?
  +-- yes --> --backend apalache  (symbolic, handles millions of states)

Just hunting for shallow bugs fast?
  +-- yes --> --backend z3 --bmc-depth 20  (finds bugs in seconds)

CI regression check?
  +-- yes --> --backend z3 --bmc-depth 30  (fast, catches most regressions)
```

10.4 BFS backend (default — learning mode)

The built-in breadth-first search over your spec's reachable states. No external tools needed. This is where you start.

```
spectre verify counter.spec
# safe -- 11 states explored (exhaustive)

spectre verify counter.spec --max-states 50000
# safe -- 11 states explored (exhaustive)
```

Best for: - Writing and iterating on specs — zero setup, immediate feedback - Specs with up to ~50k reachable states - Learning how your spec behaves

Keep in mind: - BFS explores traces; liveness results are exploratory. For validated liveness, use `--backend tlc`. - Memory-bound on large specs (> 100k states gets slow)

10.5 TLC backend

Translates your spec to TLA+ and runs it through TLC — the reference TLA+ model checker.

```
# Install TLA+ tools
brew install tla-tools          # macOS
# or download tla2tools.jar from https://github.com/tlaplus/tlaplus/releases

# Run with TLC
spectre verify mutex.spec --backend tlc
spectre verify mutex.spec --backend tlc --tlc-jar ~/tools/tla2tools.jar
```

```
Verifying mutex.spec with backend tlc...
mutex.spec: safe (exhaustive)
  Note: TLC checked all reachable states
```

Best for: - Specs with `WF(action)` or `SF(action)` — TLC validates fairness correctly using SCC-based cycle detection - Final sign-off on liveness properties - Cross-checking a spec against the TLA+ ecosystem

Limitations: - Slower than BFS for small specs - Requires Java + TLA+ tools installed

10.5.1 What Spectre generates

For each `.spec` file, Spectre creates two temp files:

```
---- MODULE myspec ----
EXTENDS Integers, TLC

VARIABLES balance, status

vars == <<balance, status>>

Init == /\ balance = 0
```

```
        /\ status  = "Active"

Deposit == /\ status = "Active"
           /\ balance < 1000
           /\ balance' = balance + 100
           /\ UNCHANGED <<status>>

Next == Deposit \/ Withdraw \/ Freeze \/ Unfreeze

Spec == Init /\ [][Next]_vars /\ WF_vars(Deposit)
====
```

```
INIT Init
NEXT Next
INVARIANT solvent
```

10.6 Apalache backend

Apalache is a symbolic model checker for TLA+. Instead of enumerating states, it uses Z3 internally to reason about sets of states at once.

```
# Install Apalache
brew install apalache
# or download from https://github.com/informalsystems/apalache/releases

# Run with Apalache
spectre verify large-spec.spec --backend apalache
spectre verify large-spec.spec --backend apalache \
    --apalache-bin /opt/apalache/bin/apalache-mc
```

Best for: - Large state spaces that exhaust BFS memory - Specs with many integer variables (Apalache reasons symbolically over integers) - Production-grade safety verification

Limitations: - Fairness (WF/SF): limited — Apalache has partial temporal support but does not perform full SCC-based liveness checking. Use TLC for validated fairness. - Slowest to start (JVM startup + type checking) - Requires Apalache installed (Java 17+)

10.6.1 BFS vs Apalache on a large spec

```
spectre verify raft.spec --max-states 100000
# bounded -- hit state limit (100000 states)

spectre verify raft.spec --backend apalache
# safe (exhaustive)
# Note: Apalache checked all reachable states symbolically
```

10.7 Z3 backend (bounded model checking)

Encodes your spec as an SMT-LIB2 query and asks Z3 to find a violation within N steps.

```
# Install Z3
brew install z3

# Default depth (20)
spectre verify bank.spec --backend z3

# Deeper search
spectre verify bank.spec --backend z3 --bmc-depth 50
```

When a violation exists within the depth:

```
Verifying bank.spec with backend z3...
bank.spec: violation found
  Note: bounded model checking to depth 20
  Violation 1: invariant "solvent"
    step 3: balance = -100
```

When no violation is found within the depth:

```
bank-fixed.spec: safe
  Note: bounded model checking to depth 20 -- not exhaustive
```

> `safe` from Z3 means **no violation up to depth 20** — not a proof of full safety. Use `--backend bfs` or `--backend apalache` for an exhaustive result.

Best for: - Finding shallow bugs quickly (< 1 second for depth 20) - CI smoke tests that should finish in a few seconds - Specs too large for BFS where you just want fast feedback

Limitations: - Bounded — can miss deep violations - No fairness checking (liveness properties not verified)

10.8 Comparison table

	BFS	TLC	Apalache	Z3
Role	learning	production	production	production
Exhaustive	small specs	yes	yes	no (bounded)
Fairness (WF/SF)	exploratory	validated	limited	none
Liveness	exploratory	validated	limited	none
Safety	exhaustive	exhaustive	exhaustive	bounded
Dependencies	none	TLA+ tools	Apalache bin	z3 binary
Speed	fast	moderate	fast	very fast
Setup	0 min	5 min	10 min	2 min

10.9 Progressive verification workflow

Start with BFS while writing. Graduate to production backends for sign-off.

```
Write spec
    |
    v
--backend bfs         <- instant feedback, no deps, great for learning
    |
  passes?
    |
    +- no --> fix spec, repeat
    |
    +- yes
         |
         v
    Has WF/SF fairness or liveness?
         +- yes --> --backend tlc   (validated SCC-based checking)
         |
         +- no, state space large?
              +- yes --> --backend apalache  (symbolic exhaustive)
              +- no  --> done  (BFS exhaustive for small specs)
```

In CI, combine:

```
# Fast regression check (seconds)
spectre verify myspec.spec --backend z3 --bmc-depth 30

# Full verification (minutes, on merge to main)
spectre verify myspec.spec --backend tlc
```

10.10 Installing the backends

Backend	Install command	Verify install
BFS	built in	`spectre --version`
TLC	`brew install tla-tools`	`tlc -help`
Apalache	`brew install apalache`	`apalache-mc version`
Z3	`brew install z3`	`z3 --version`

On Linux, download JARs/binaries from the respective GitHub releases pages and pass `--tlc-jar`, `--apalache-bin`, or `--z3-bin` flags.

11 Mining specs from Rust

> **Important limitations before you start:** The miner cannot extract enum payload data (`InsufficientFunds(u64)` becomes `InsufficientFunds` with no data), error

propagation paths (`Result<T,E>` error branches are dropped), and arithmetic safety operations (`checked_add`, `saturating_sub` are approximated as unbounded arithmetic). These are precisely the locations where real Rust bugs occur. After mining, run `--show-omissions` and add invariants or guards for every dropped case that matters for safety.

11.1 What mining does

```
src/account.rs --> spectre mine --> account.spec skeleton
                                          |
                                    add invariants
                                          |
                                    spectre verify
```

The miner reads your Rust source and extracts struct fields, assert guards, and field assignments — giving you a runnable spec in seconds.

What it extracts: - `struct` fields -> `var` declarations - `assert!` / `debug_assert!` -> `require` guards - `if cond { return Err(..) }` -> negated `require` guards - `self.field = expr` / `self.field += n` -> primed assignments - `fn new(...)` constructor -> `init` values - Rust enums -> Spectre enums

---

11.2 Your first mine

```rust
// src/counter.rs
pub struct Counter {
    pub value: i64,
    pub max:   i64,
}

impl Counter {
    pub fn new(max: i64) -> Self {
        Counter { value: 0, max }
    }
    pub fn increment(&mut self) {
        assert!(self.value < self.max);
        self.value += 1;
    }
    pub fn decrement(&mut self) {
        assert!(self.value > 0);
        self.value -= 1;
    }
    pub fn reset(&mut self) {
        self.value = 0;
    }
}
```

```
spectre mine --lang rust src/counter.rs
```

Output:

```
// Spectre spec mined from: src/counter.rs

var value: int
var max:   int

init {
  value = 0
  max   = 0
}

action increment {
  require value < max
  value' = value + 1
}

action decrement {
  require value > 0
  value' = value - 1
}

action reset {
  value' = 0
}
```

Save it and verify immediately:

```
spectre mine --lang rust src/counter.rs -o counter.spec
spectre verify counter.spec
# Exhaustive verification: traversed 1 states (full reachable space)
# Found no violations.
# (max=0 by default - set max to a real value before verifying)
```

11.3 What gets extracted vs. what gets skipped

Extracted (ok)	Skipped / approximated (x)
Primitive fields (`i64`, `bool`, `String`)	`new()`, `default()`, `clone()`, `fmt()`
Rust enums — unit variants only	Tuple/struct enum variants (data dropped)
`assert!` / `debug_assert!` guards	`panic!`, `unwrap()`, `expect()`
`self.x = …`, `self.x += …`, `self.x -= …`	`checked_add`, `wrapping_add`, `saturating_sub`
`if cond { return Err(..) }` patterns	`Result<T,E>` error paths

Extracted (ok)	Skipped / approximated (x)
`Arc<T>`, `Mutex<T>`, `Box<T>` wrappers	(inner type extracted, wrapper dropped)

11.4 Seeing what was skipped: `--show-omissions`

```
spectre mine --lang rust src/account.rs --show-omissions
```

```
Extraction omissions (4):
  SKIPPED_METHOD in new: constructor/utility method - not translated to action
  RESULT_RETURN in deposit: Result<(),TxError> return - error path abstracted
  OVERFLOW_OP in deposit: checked_add - saturating arithmetic abstracted
  TUPLE_VARIANT in TxError: InsufficientFunds(i64) - data dropped, unit variant kept

Mined: 3 fields, 1 enum, 4 actions, 4 omissions
```

Four omission kinds to act on:

- **RESULT_RETURN** — error paths not modelled. Add invariants for the error conditions manually if they matter.
- **OVERFLOW_OP** — arithmetic is unbounded in the spec. Add `require` guards if overflow-safe bounds are important.
- **TUPLE_VARIANT** — enum variant data was dropped. Add a `var` for the data if you need to reason about it.
- **SKIPPED_METHOD** — constructors, `fmt`, `drop`. Almost always safe to ignore.

The mine summary line shows how many omissions occurred:

```
Mined: 3 fields, 1 enum, 4 actions, 4 omissions (run --show-omissions for details)
```

11.5 Evaluate a whole directory

```
spectre eval-rust --dir src/
```

```
# Rust Extraction Evaluation Report

## Per-File Summary

| File              | Struct      | Fields | Enums | Methods | Omissions | Success |
|-------------------|-------------|--------|-------|---------|-----------|---------|
| account.rs        | BankAccount |   3    |   1   |    4    |     4     |   50%   |
| counter.rs        | Counter     |   2    |   0   |    3    |     1     |  100%   |
| state_machine.rs  | Machine     |   4    |   2   |    2    |     4     |   50%   |

**Totals** - files: 3 | fields: 9 | methods: 9 | omissions: 9 | mean success: 67%

## Omissions by Kind
```

Kind	Count
SKIPPED_METHOD	3
RESULT_RETURN	2
OVERFLOW_OP	2
TUPLE_VARIANT	2

Success fraction = methods with no body-level omissions (overflow / panic / Result / complex assign). Files at 100% can be verified with minimal manual work. Files at 50% need review.

Use this output to prioritise enrichment effort — start with files nearest 100%.

11.6 Enrich the mined spec

Three things to always add after mining:

1. **Invariants** — the miner extracts no invariants (only guards). Add at least one per resource, one per ordering constraint.
2. **Temporal properties** — if your system makes progress guarantees, add `WF(action)` or `eventually(goal)`.
3. **Realistic init values** — `max = 0` won't verify anything useful. Set values that match production defaults.

Example enrichment for the counter spec:

```
// counter.spec - after enrichment
var value: int
var max:   int

init {
  value = 0
  max   = 10    // <- set a realistic default
}

action increment {
  require value < max
  value' = value + 1
}

action decrement {
  require value > 0
  value' = value - 1
}

action reset {
  require value > 0    // guard prevents stutter when value is already 0
  value' = 0
}
```

```
// <- these three lines are the additions
invariant non_negative  { value >= 0 }
invariant within_bounds { value <= max }
temporal  progress      { WF(increment) }
```

11.7 LLM-enhanced mining

```
ANTHROPIC_API_KEY=sk-... spectre mine --lang rust src/account.rs --ai
```

The LLM adds (always commented out — never applied automatically): - One-sentence `description` for each action - Up to three `invariant` candidates - Up to three `temporal` property candidates

```
// LLM-suggested invariants (review before enabling):
// invariant balanced_books { total_deposits >= total_withdrawals }
// invariant no_freeze_with_balance { frozen -> balance = 0 }
```

Always review LLM suggestions against your actual requirements before uncommenting.

11.8 The full mining workflow

```
1. spectre eval-rust --dir src/
   +-- identify which files mine cleanly (Success = 100%)

2. spectre mine --lang rust src/target.rs -o target.spec
   +-- get the skeleton

3. spectre mine ... --show-omissions
   +-- understand what was skipped

4. Edit target.spec
   +-- add invariants, fix init values, handle omissions

5. spectre verify target.spec
   +-- safe -> check in both files
       violation -> fix spec, re-verify

6. spectre sync src/target.rs --spec target.spec  (ongoing)
   +-- catch drift as code evolves
```

12 Drift detection and incremental re-verification

12.1 The drift problem

```
Day 1:   mine spec --> check in account.spec + account.rs
              |
```

```
Day 30:  refactor withdraw() in Rust
              |
              v
         spec and code no longer agree  <- silent drift
              |
         next production incident reveals it
```

`spectre sync` catches drift before it becomes a bug report.

12.2 Running sync

```
spectre sync src/account.rs --spec account.spec
```

Each action gets one of three symbols:

Symbol	Meaning
`[=]`	Unchanged — guards and assignments are structurally identical
`[=]`	SMT-proved equivalent — code changed, semantics identical, no re-verify needed
`[x]`	Semantically changed — SMT found a counterexample; re-verify required

Example output after a bug is introduced in `withdraw`:

```
[=]  action deposit    (structurally unchanged)
[x]  action withdraw   assignment 1 body changed:
       "balance' = balance - amount"  ->  "balance' = balance + amount"
       witness: amount=1, balance=0
[=]  action freeze     (structurally unchanged)
[=]  action unfreeze   guard rewritten (SMT-proved equivalent):
       "!frozen"  ->  "frozen == false"
```

The `[x]` line gives you: - **What changed** — which assignment or guard - **The witness** — concrete values where the two versions behave differently

The SMT-equivalence check prevents false alarms on equivalent rewrites — commutativity, constant folding, guard commutation.

12.3 CI integration

```yaml
# .github/workflows/spectre.yml
- name: Check for spec drift
  run: spectre sync src/account.rs --spec account.spec
  # exits non-zero if any [x] is found
```

`sync` exits 0 if all actions are `[=]` (unchanged or SMT-equivalent); exits non-zero on any semantic change. Safe to gate a merge on.

12.4 Incremental re-verification

After a semantic change, you don't need to re-run full BFS. Re-verify only the changed action:

```
spectre verify account.spec \
  --incremental \
  --changed-action withdraw \
  --use-cache
```

How it works (5 steps):

1. Load the cached state graph from the last full run
2. Remove all states reachable only through `withdraw`
3. Re-explore from states where `withdraw` was previously enabled
4. Re-check invariants on the new states only
5. Merge result back into the cache

Output:

```
Incremental re-verification: action 'withdraw'
  States re-explored: 12  (vs 847 in full BFS)
  safe -- no new violations
  Cache updated.
```

12 states vs 847 = **70x faster** for this example. The saving grows with spec size.

Soundness note: Incremental re-verification is sound when the changed action's new transition relation is a subset of its old one — strengthened guards, narrowed effects. If the action gained new behaviors (new states it can reach), the incremental check may miss violations introduced through alternative cycles that the previous full run never explored. When in doubt, run a full re-verification: `spectre verify account.spec`.

12.5 Impact analysis

Before re-verifying, check which invariants are actually affected by the change:

```
spectre impact account.spec
```

```
Changed actions (from git diff):
  withdraw -- semantics changed (SMT witness: amount=1, balance=0)

Affected invariants:
  solvent         <- balance can go negative with the new body
  not_over_limit  <- not directly affected by withdraw

Recommended:
  spectre verify account.spec --incremental --changed-action withdraw
```

Use `impact` in CI to skip re-verification when a changed action doesn't touch any safety-critical invariant.

12.6 Full sync workflow

```
Code changes committed
        |
        v
spectre sync src/ --spec spec.spec
        |
  [=] only?
        +-- yes --> nothing to do
        +-- no ([x] found)
                |
                v
        spectre impact spec.spec
                |
        safety-critical invariant affected?
                +-- no  --> log warning, skip re-verify
                +-- yes
                        |
                        v
                spectre verify spec.spec \
                  --incremental \
                  --changed-action NAME \
                  --use-cache
                        |
                   safe? --yes--> commit + update cache
                        |
                        no
                        |
                        v
                Fix code or spec, re-verify
```

12.7 Sync evaluation: what it checks

`sync` compares: - Struct field names and types - Action guard counts and expressions - Assignment-body expressions (normalised)

It detected **30/35 injected semantic mutations** (85.7%) in evaluation, with 1 false positive (vs 4 with AST comparison alone). The SMT-equivalence classification eliminates the majority of false alarms.

12.8 Tips for keeping sync useful

- **Check in the spec file alongside the Rust source** — keep them in the same directory or a `spec/` subdirectory
- **Run sync in pre-commit** as well as CI — catch drift before it even reaches the repo
- **After a semantic change**: always re-verify before merging, even if you think the change is safe
- **After an SMT-equivalent rewrite**: no action required — but it's worth reading the rewrite to confirm it's intentional

13 Model-based testing

13.1 The idea

```
account.spec
      |
      v
spectre verify --emit-traces trace.itf.json
      |
      v
spectre generate-driver --lang rust account.spec -o src/driver.rs
      |
      v
fill in apply_action() + current_state() against your BankAccount impl
      |
      v
cargo test
      |
      +-- (ok) 6/6 steps match -- implementation agrees with spec
      +-- (x) step 3 mismatch  -- spec found a bug in your Rust code
```

The spec is the oracle. The driver replays spec-generated execution traces against your actual Rust implementation and reports any disagreement.

13.2 Generate a trace

```
spectre verify account.spec \
  --emit-traces trace.itf.json \
  --coverage-mode property
```

Coverage modes — pick based on what you want to exercise:

Mode	Strategy	Best for
`action`	Prefer actions not yet seen in trace	Breadth of action coverage (default)
`property`	Steer toward invariant boundary values	Catching off-by-one bugs

Mode	Strategy	Best for
`transition-pair`	Cover all (prev, next) action pairs	Transition sequence coverage
`boundary`	Maximise integer variable values	Overflow and limit bugs
`rare-action`	Prefer globally least-executed actions	Obscure code paths

`property` mode achieves 52% boundary-state coverage vs 5--25% for the others — use it when chasing subtle arithmetic bugs.

13.3 Generate the driver

```
spectre generate-driver --lang rust account.spec -o src/driver.rs
```

`driver.rs` gives you two traits to implement:

```rust
pub trait AccountDriver {
    /// Apply one action from the spec trace to your implementation.
    fn apply_action(&mut self, action: &str, args: &[Value]) -> Result<(), String>;

    /// Return the current implementation state in spec terms.
    fn current_state(&self) -> SpecState;
}
```

`SpecState` mirrors the spec's `var` declarations:

```rust
pub struct SpecState {
    pub balance: i64,
    pub frozen:  bool,
}
```

Wire these to your `BankAccount` struct and you're done.

13.4 CI-ready driver with `--ci`

```
spectre generate-driver --lang rust account.spec --ci -o src/driver.rs
```

Three files are written:

File	What to do with it
`src/driver.rs`	Implement the two trait methods
`src/driver.cargo.toml`	Paste the `[dev-dependencies]` block into `Cargo.toml`
`src/driver.ci.yml`	Copy to `.github/workflows/` — runs `cargo test --test driver` on every push

The Cargo snippet:

```
# paste into your [dev-dependencies]
spectre-connect = { git = "https://github.com/BoundlessProducts/spectre-connect" }
```

The CI workflow:

```
name: Spectre conformance
on: [push, pull_request]
jobs:
  conformance:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: rust-toolchain@stable
      - run: cargo test --test driver
```

No extra configuration — the workflow picks up `trace.itf.json` from the repo.

13.5 Run the replay

```
cargo run -- trace.itf.json
```

Passing run:

```
Replaying 6 steps...
  step 1  deposit(100):   balance  0 -> 100  (ok)
  step 2  deposit(100):   balance 100 -> 200  (ok)
  step 3  withdraw(50):   balance 200 -> 150  (ok)
  step 4  freeze:         frozen false -> true  (ok)
  step 5  deposit(100):   skipped (frozen)  (ok)
  step 6  unfreeze:       frozen true -> false  (ok)
(ok) 6/6 steps match
```

Failing run:

```
(x) 1 mismatch:
  step 3, action 'withdraw', var 'balance':
    spec expects: 150
    impl returned: 200
    hint: check withdraw() arithmetic - possible sign error
```

13.6 Violation traces as regression tests

When `spectre verify` finds a violation, `--emit-traces` captures the exact counterexample path:

```
spectre verify account.spec --emit-traces violation.itf.json
```

Check this file into your repo. It should: - **Fail** against the unfixed implementation - **Pass** after you apply the fix

```
# Before fix:
cargo run -- violation.itf.json  # -> (x) mismatch at step 4

# After fix:
cargo run -- violation.itf.json  # -> (ok) 4/4 steps match
```

This turns every spec violation into a permanent, reproducible regression test.

13.7 Repair suggestions before fixing

When verification fails, get candidate patches before editing anything:

```
spectre verify account.spec --suggest-repairs
```

```
Repair candidates (CANDIDATE - review before applying):

Violation 1: action 'withdraw' violates 'solvent'
  [1] CANDIDATE strengthen_require: add `require balance >= amount` to withdraw
      Validation: violation fixed . no deadlock . reachability unchanged - SAFE
  [2] CANDIDATE add_invariant_guard: add `when balance >= amount` to withdraw
      Validation: violation fixed . deadlock at balance=0 - review required
```

How to pick:

Candidate type	Pick when	Risk
`strengthen_require`	The action should simply not fire when condition fails	Minimal
`add_invariant_guard`	You want the action blocked at the invariant boundary	Can introduce deadlock

Apply the chosen patch to the spec, re-verify, then propagate the change to the Rust source.

13.8 Checking refinement

When you have traces from an external source (a different tool, a recorded production run), check whether they conform to your spec:

```
spectre check-refinement account.spec
```

This reads `driver.json` (the refinement mapping) and validates that each external trace is a valid execution of the spec.

13.9 Complete MBT workflow

```
1.  spectre mine --lang rust src/account.rs -o account.spec
          |
```

```
2.  Add invariants + temporal properties to account.spec
          |
3.  spectre verify account.spec
          +-- violation? -> --suggest-repairs -> fix -> repeat
          +-- safe
                |
4.  spectre verify account.spec \
      --emit-traces trace.itf.json \
      --coverage-mode property
          |
5.  spectre generate-driver --lang rust account.spec --ci -o src/driver.rs
          |
6.  Implement apply_action() and current_state() in driver.rs
          |
7.  cargo test --test driver
          +-- mismatch? -> read step output -> fix Rust impl -> repeat from 7
          +-- (ok) all pass
                |
8.  Check in: account.spec, trace.itf.json, driver.rs, driver.ci.yml
          |
9.  On every code change:
      spectre sync src/account.rs --spec account.spec
          +-- [(x)] found? -> re-verify -> update trace -> repeat from 7
          +-- [=]/[=] -> nothing to do
```

14 Runtime monitoring

Formal verification proves your spec is safe. A runtime monitor checks your *running application* against those same spec invariants — catching violations that formal verification cannot see (third-party library updates, misconfigured deployments, out-of-spec inputs).

14.1 The gap formal verification leaves

```
spectre verify account.spec
        |
   safe -- all reachable states satisfy invariants
        |
   ship to production
        |
   third-party library update changes behaviour  <- not in your spec
        |
   invariant violated at runtime -- never detected
```

A runtime monitor closes this gap by re-checking the spec's invariants on every real state transition.

14.2 What the generated monitor actually is

`spectre generate-monitor` produces a Rust struct with one inline boolean assertion per spec invariant. It is a **generated assertion library** — not a formal runtime verifier. Its value is that the assertions are derived from the exact same predicates your model checker verified, so they are guaranteed to be consistent with the spec.

The `SpecState` mapping between your application and the monitor is **entirely on you**. If you map the wrong fields, the monitor checks the wrong things.

14.3 Generate a monitor

```
spectre generate-monitor --lang rust account.spec -o src/monitor.rs
```

The generated `monitor.rs` gives you:

```rust
/// Mirrors the spec variables. You fill this in from your application state.
#[derive(Debug, Clone)]
pub struct SpecState {
    pub balance: i64,
    pub frozen:  bool,
}

/// A spec invariant violation detected at runtime.
#[derive(Debug)]
pub struct Violation {
    pub step:      u64,
    pub invariant: &'static str,
    pub state:     SpecState,
}

pub struct Monitor { /* private */ }

impl Monitor {
    pub fn new() -> Self { Monitor::default() }

    /// Call after every state transition. Checks all spec invariants.
    /// Returns Some(Violation) if any invariant fails.
    pub fn step(&mut self, state: &SpecState) -> Option<Violation> { ... }

    /// Sample liveness properties periodically (e.g. every 1000 steps).
    pub fn check_liveness(&self) -> Vec<&'static str> { ... }
}
```

Each `step()` call is O(k) where k is the number of invariants — typically two to five integer comparisons, with no heap allocation.

14.4 Wire it into your application

```rust
use crate::monitor::{Monitor, SpecState};

struct App {
    account: BankAccount,
    monitor: Monitor,
}

impl App {
    fn deposit(&mut self, amount: i64) {
        self.account.deposit(amount);

        // You are responsible for this mapping being correct.
        let state = SpecState {
            balance: self.account.balance,
            frozen:  self.account.frozen,
        };

        if let Some(v) = self.monitor.step(&state) {
            tracing::error!(
                "spec violation: {} at step {}",
                v.invariant, v.step
            );
            // handle: log, alert, rollback, or panic -- your choice
        }
    }
}
```

14.5 Handle violations your way

Response	When to use
Log + continue	Non-critical invariant, high-traffic path
Alert + continue	Production monitoring with async alerting
Panic	Safety-critical — prefer crash over corrupt state

14.6 Validate monitor traces before deploying

```
spectre validate-monitor account.spec
```

```
Validated 2 violation trace(s):
  [1] action 'withdraw' -> invariant 'solvent': consistent (4 steps, final violated)
  [2] action 'deposit'  -> invariant 'within_cap': consistent (3 steps, final violated)

2/2 consistent.
```

A **consistent** trace means the BFS path genuinely leads to a state that fails the target invariant. Run `validate-monitor` after every spec change before regenerating the monitor.

14.7 What the monitor checks — and what it doesn't

Checks: - Every `invariant` from the spec, after every `step()` call - Liveness properties when you call `check_liveness()`

Does not check: - Whether your `SpecState` mapping is correct — that is entirely on you - Temporal properties in real-time (sampled only on `check_liveness`) - Invariants over variables you did not include in `SpecState`

14.8 Full monitoring workflow

```
account.spec
     |
     v
spectre verify account.spec
     |  safe -- design-time exhaustive check
     v
spectre validate-monitor account.spec
     |  all violation traces consistent
     v
spectre generate-monitor --lang rust account.spec -o src/monitor.rs
     |
     v
implement SpecState mapping in application
     |
     v
call monitor.step(&state) after every transition
     |
     v
deploy
     |
     +-- violation fires --> log / alert / panic per policy
     +-- no violation    --> spec holds in every observed execution
```

14.9 Performance overhead

For `account.spec` with two invariants, each `step()` call is two integer comparisons:

```
~2 ns per step (two invariants, no allocation)
```

At 100,000 transitions per second that is 0.02% CPU overhead — negligible in any real workload.

15 Benchmarking and performance

Before you change a spec, you want to know how big the state space is and how fast Spectre explores it. That is what the built-in benchmark suite is for.

15.1 Run the benchmarks

```
spectre benchmark
```

```
Benchmark results (BFS backend)
================================
counter.spec        10 states   0.12 ms    depth 5
account.spec        84 states   0.41 ms    depth 8
mutex.spec         430 states   2.1 ms     depth 14
dist_queue.spec   3210 states   18 ms      depth 24
ring_leader.spec  9980 states   94 ms      depth 40

Total: 5 specs, 13714 states, 115 ms
```

The benchmark runs your spec files in `benchmarks/` against the BFS backend and reports state count, wall time, and max depth.

Add your own specs:

```
# drop any .spec file into benchmarks/
cp account.spec benchmarks/

spectre benchmark
```

15.2 Understand state-space growth

State spaces grow exponentially with the number of variables and their ranges. Here is what to expect:

Pattern	Typical state count	Why
Counter (1 var, range 0-10)	~11	Linear
2 independent counters (0-10 each)	~121	Product
Mutex (2 threads, 3 states each)	~9	Small product
Mutex + shared counter (0-10)	~90	3-way product
Producer-consumer (queue depth 5)	~500	Queue contents combinatorial

Pattern	Typical state count	Why
Ring leader election (5 nodes)	~100K+	Permutation space

The rule: **every new variable multiplies the state space by its domain size**.

15.3 State-space growth diagram

```
Variables:  1     2     3     4     5
            |     |     |     |     |
Range 10:  10   100  1000  10K  100K
Range 5:    5    25   125  625   3K
Range 2:    2     4     8   16   32
            |
            +-- BFS exhausts this in < 1 ms
                              |
                              +-- starts taking seconds
                                          |
                                          +-- needs --backend tlc or z3
```

15.4 Bounded exploration options

When a spec is too large for exhaustive BFS, cap the search:

```
# Cap at 50,000 states
spectre verify --max-states 50000 ring_leader.spec

# Cap at depth 30
spectre verify --max-depth 30 ring_leader.spec

# Both
spectre verify --max-states 50000 --max-depth 30 ring_leader.spec
```

The result will say `(bounded)` rather than `(exhaustive)`.

15.5 Partial-order reduction

For concurrent systems, many interleavings produce identical states. `--por` (partial-order reduction) prunes these by exploring only one representative ordering:

```
spectre verify --por mutex.spec
```

```
Before --por: 430 states, 2.1 ms
After  --por: 127 states, 0.7 ms
```

Savings are largest when: - Many threads share little state - Actions are mostly independent

`--por` is sound for safety properties. Avoid it for liveness properties unless you also use `--backend tlc`.

15.6 Property-guided exploration

```
spectre verify --property-guided mutex.spec
```

Property-guided mode biases BFS toward states that are close to violating an invariant. It finds bugs faster on large specs but may miss violations in rarely-reached corners.

Use it for: - Interactive debugging — "show me a violation fast" - Large specs where exhaustive BFS times out

Do not use it for production sign-off — use exhaustive BFS or TLC for that.

15.7 When to switch backends

Situation	Recommendation
Learning or iterating on a spec	BFS (default, zero setup)
Safety only, <= 100K states	BFS
Fairness / liveness properties	`--backend tlc` (validated SCC-based)
> 1M states, safety only	`--backend apalache` (symbolic)
Fast shallow bug hunting (< 20 steps)	`--backend z3 --bmc-depth 20`
Exhaustive + fairness + large space	`--backend tlc`

15.8 Progressive verification strategy

Start cheap, escalate only if needed:

```
spectre verify --max-states 10000 my.spec   (fast exploration)
       |
   safe in 10K states --> probably fine for small specs
       |
   violation or exhausted
       |
       v
spectre verify my.spec                          (exhaustive BFS, no limit)
       |
   exhaustive safe --> ship
       |
   too large / times out
       |
       v
```

```
spectre verify --backend z3 --bmc-depth 50 my.spec   (fast shallow bugs)
        |
   sat -> violation in <=50 steps --> fix, restart
        |
   unsat (no shallow bugs found)
        |
        v
spectre verify --backend tlc my.spec                 (full model check)
```

Each step is significantly more expensive. Stop as soon as you get a definitive answer.

15.9 Benchmark in CI

```yaml
# .github/workflows/ci.yml (excerpt)
- name: Benchmark smoke test
  run: |
    spectre benchmark --max-states 1000 --ci
    # Fails if any spec exceeds 1000 states (catches accidental bloat)
```

The `--ci` flag exits non-zero if any spec exceeds `--max-states`. This catches spec bloat before it becomes a slow-CI problem.

16 TLA+ and Apalache backend

BFS is the learning backend — great for writing and exploring specs. When you need validated fairness semantics or your state space is too large for BFS, `--backend tlc` and `--backend apalache` hand off to proven model checkers built on TLA+.

16.1 Quick start

```
# Run TLC model checker
spectre verify --backend tlc counter.spec

# Run Apalache symbolic model checker
spectre verify --backend apalache counter.spec

# Point to non-default TLC jar
spectre verify --backend tlc --tlc-jar /opt/tlc/tla2tools.jar counter.spec

# Point to non-default Apalache binary
spectre verify --backend apalache --apalache-bin /usr/local/bin/apalache-mc counter.spec
```

16.2 How Spectre translates to TLA+

Spectre translates your `.spec` file into a standard TLA+ module plus a `.cfg` file. Here is the counter example:

`counter.spec`

```
var x: int
param MAX: int
init { x = 0 }
action incr { require x < MAX  x' = x + 1 }
invariant non_negative { x >= 0 }
temporal wf_progress { WF(incr) }
```

Generated `counter.tla`

```
--------------------------- MODULE counter ---------------------------
EXTENDS Naturals, Sequences, TLC

CONSTANTS MAX

VARIABLES x

vars == <<x>>

TypeOK == x \in Int

Init ==
    x = 0

incr ==
    /\ x < MAX
    /\ x' = x + 1

Next ==
    \/ incr

non_negative ==
    x >= 0

Spec ==
    Init /\ [][Next]_vars /\ WF_vars(incr)

============================================================================
```

Generated `counter.cfg`

```
INIT Init
NEXT Next
INVARIANT non_negative
PROPERTY Spec
```

```
MAX = 100
```

Then Spectre runs `tlc counter.tla -config counter.cfg` and parses the output.

16.3 Expression mapping reference

Spectre	TLA+
`x != y`	`x # y`
`x <= y`	`x =< y`
`x && y`	`x /\ y`
`x || y`	`x \/ y`
`!x`	`~x`
`true`	`TRUE`
`false`	`FALSE`
`always(P)`	`[](P)`
`eventually(P)`	`<>(P)`
`WF(action)`	`WF_vars(action)` in `Spec`
`SF(action)`	`SF_vars(action)` in `Spec`
`x'`	`x'`
`UNCHANGED x, y`	`UNCHANGED <<x, y>>`

16.4 Why TLC for fairness?

BFS explores traces and flags cases where an enabled action is never taken. This is exploratory — useful for learning, but not an authoritative liveness result.

TLC's approach is exact:

```
Liveness check (4 steps):

1. Decompose Spec into safety + fairness constraints
   Spec == Init /\ [][Next]_vars /\ WF_vars(incr)
                                    +-- fairness constraint

2. Find all SCCs in the state graph
   (strongly connected components = potential infinite loops)

3. For each SCC, check: does incr get to fire?
   If incr is enabled in some state of an SCC but never fires
   -> genuine fairness violation

4. Report: the specific cycle that violates WF(incr)
```

This SCC-based approach is why `--backend tlc` gives you authoritative liveness results.

16.5 TLC vs Apalache: when to use which

	TLC	Apalache
Approach	Explicit-state enumeration	Symbolic (SMT-based)
Best for	Liveness, fairness, small-medium spaces	Large state spaces, safety
Fairness (WF/SF)	Validated (exact SCC-based)	Limited
Temporal / liveness	Validated	Limited
Safety invariants	Exhaustive	Exhaustive (symbolic)
State space limit	~10M states practical	Effectively unbounded
Speed on small specs	Fast	Slower (SMT overhead)
Speed on large specs	Slow (memory)	Fast (symbolic)
Counterexample	Full trace	Symbolic witness
Requires Java	Yes	Yes

Rule of thumb: - Use **TLC** when you have `temporal` or `WF/SF` properties — it is the only backend that validates fairness correctly - Use **Apalache** when BFS times out on safety properties — it handles large spaces symbolically, but fairness support is limited

---

16.6 Pass parameters to TLC

```
spectre verify --backend tlc --param MAX=100 --param THREADS=4 mutex.spec
```

Parameters become TLC `CONSTANTS` assignments in the `.cfg` file:

```
CONSTANTS
MAX = 100
THREADS = 4
```

16.7 Read TLC output

When TLC finds a violation, Spectre parses the trace and prints it in the same format as BFS:

```
mutex.spec: violation found
  Violation 1: invariant "mutual_exclusion" triggered by action "enter_cs"
    Step 0: [pc_a = idle,    pc_b = idle,    flag_a = false, flag_b = false]
    Step 1: [pc_a = wanting, pc_b = idle,    flag_a = true,  flag_b = false]
    Step 2: [pc_a = wanting, pc_b = wanting, flag_a = true,  flag_b = true ]
```

```
    Step 3: [pc_a = critical,pc_b = wanting, flag_a = true,  flag_b = true ]
    Step 4: [pc_a = critical,pc_b = critical,flag_a = true,  flag_b = true ]
                                              ^ both in critical section -- violated
```

16.8 TLC installation

TLC comes bundled in the `tla2tools.jar` distributed by the TLA+ project:

```
# macOS (Homebrew)
brew install tla-tools

# Or download tla2tools.jar manually and point Spectre at it:
spectre verify --backend tlc --tlc-jar ~/tools/tla2tools.jar account.spec
```

Spectre tries `tlc` on `$PATH` first, then falls back to `java -jar <tlc-jar>`.

16.9 Apalache installation

```
# macOS
brew install apalache

# Or set the binary path:
spectre verify --backend apalache --apalache-bin ~/tools/apalache/bin/apalache-mc account.spec
```

17 Z3 bounded model checking

The Z3 backend is your fastest path to finding shallow bugs. Instead of exploring the state graph step by step, it encodes your spec as an SMT-LIB2 formula and asks Z3 a single question: *is there any execution of length <= N that violates an invariant?*

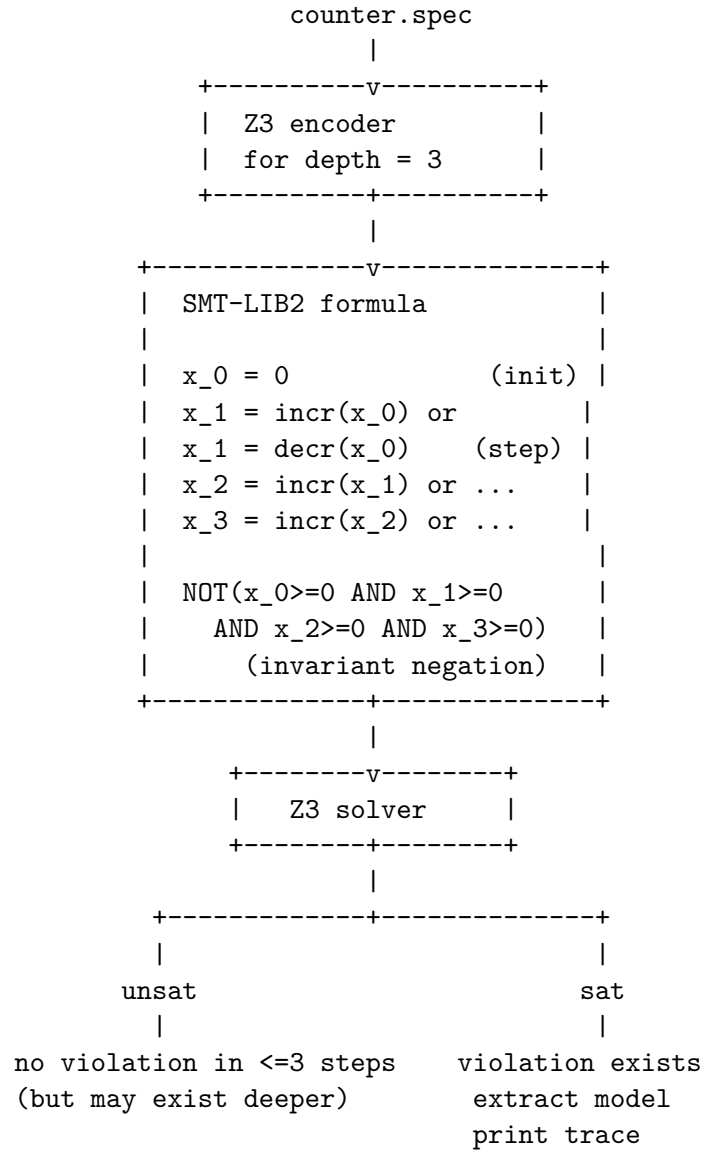
17.1 Quick start

```
# Check for violations in up to 20 steps (default depth)
spectre verify --backend z3 counter.spec

# Increase depth for longer paths
spectre verify --backend z3 --bmc-depth 50 account.spec

# Point to non-default Z3 binary
spectre verify --backend z3 --z3-bin /usr/local/bin/z3 account.spec
```

17.2 How bounded model checking works



17.3 SMT-LIB2 encoding example

For this spec:

```
var x: int
init { x = 0 }
action incr { require x < 5  x' = x + 1 }
action decr { require x > 0  x' = x - 1 }
invariant non_negative { x >= 0 }
```

Spectre generates (simplified):

```
; Step-indexed variables
```

```
(declare-const x_0 Int)
(declare-const x_1 Int)
(declare-const x_2 Int)

; Initial state
(assert (= x_0 0))

; Step 0 -> 1: one action fires, or stutter (no action enabled)
(assert (or
  (and (< x_0 5) (= x_1 (+ x_0 1)))  ; incr fires
  (and (> x_0 0) (= x_1 (- x_0 1)))  ; decr fires
  (= x_1 x_0)                          ; stutter: no action enabled
))

; Step 1 -> 2
(assert (or
  (and (< x_1 5) (= x_2 (+ x_1 1)))
  (and (> x_1 0) (= x_2 (- x_1 1)))
  (= x_2 x_1)                          ; stutter
))

; Invariant negation: find ANY step that violates non_negative
(assert (or
  (not (>= x_0 0))
  (not (>= x_1 0))
  (not (>= x_2 0))
))

(check-sat)
(get-model)
```

The stutter clause `(= x_{k+1} x_k)` is essential. Without it, a deadlocked state — one where all action guards are false — would produce `(assert (or false false))` = `(assert false)`, making the entire formula unsatisfiable. Z3 would then report "safe" even if the invariant is violated in the deadlocked state. The stutter ensures the system can always stay put, matching TLA+'s standard semantics.

Z3 answers `unsat` (no violation in 2 steps) or `sat` (violation found, model gives the path).

17.4 Read Z3 output

No violation found (up to depth):

```
counter.spec: safe (z3, bounded -- depth 20)
  Note: no violation in <=20 steps; deeper paths not checked
```

Violation found:

```
account.spec: violation found (z3)
```

```
Violation 1: invariant "solvent" triggered by action "withdraw"
  x_0 = balance:100
  x_1 = balance:50  (deposit 50 -- now 100)
  x_2 = balance:-10 (withdraw 60 -- violated)
```

17.5 When to use Z3 BMC

Good fit: - You want to find shallow bugs fast during development - Your state space is too large for exhaustive BFS - You have many variables but expect violations within ~50 steps - You want a quick sanity check before a full TLC run

Poor fit: - You need an exhaustive proof of safety (Z3 BMC can only say "no bug in N steps") - You have liveness or fairness properties (Z3 BMC checks only reachability) - Violations require very long paths (rare in practice — most bugs appear early) - Your spec uses `Set<T>` or `Map<K,V>` with `.size()` comparisons — Z3 set cardinality over integer domains is unreliable; use TLC for those specs

`init oneOf` with Z3: When your spec uses `init oneOf { ... }` to declare multiple starting states, Z3 encodes each option as an `(assert (or ...))` disjunction at step 0. This is sound — Z3 explores violations starting from any of the listed initial states. BFS and TLC handle `oneOf` natively as separate starting points.

17.6 Choose the right depth

Scenario	Suggested `--bmc-depth`
Quick sanity check	10-20
Standard development	20-50
Protocol with long setup phase	50-100
Deep corner cases	100-200
> 200	Consider TLC/Apalache instead

Doubling the depth roughly doubles the formula size and Z3 solve time. Past depth 200 you are usually better off with TLC or Apalache.

17.7 Z3 vs other backends

	BFS	Z3 BMC	TLC	Apalache
Role	learning / explore	production (bounded)	production	production
Exhaustive	small specs	no (bounded)	yes	yes (symbolic)
Fairness (WF/SF)	exploratory	none	validated (SCC)	limited

	BFS	Z3 BMC	TLC	Apalache
Temporal / liveness	exploratory	none	validated	limited
Safety invariants	exhaustive (small)	bounded	exhaustive	exhaustive (symbolic)
Speed on shallow bugs	fast	fastest	slow	medium
Large state space	no	yes	no	yes
Counterexample	full path	full path (<=N)	full path	symbolic
External dependency	none	z3 binary	Java+TLC	Java+Apalache

17.8 Progressive verification with Z3

Z3 BMC fits naturally into a staged workflow:

```
spectre verify --backend z3 --bmc-depth 20 my.spec
        |
   unsat (no bug <=20 steps)
        |
        v
spectre verify --backend z3 --bmc-depth 100 my.spec
        |
   unsat (no bug <=100 steps)
        |
        v
spectre verify my.spec          (exhaustive BFS if state space allows)
        |
        v
spectre verify --backend tlc my.spec    (for liveness / large space)
```

Each stage gives you increasing confidence. Stop as soon as you get a definitive result or a violation to fix.

17.9 Z3 installation

```
# macOS (Homebrew)
brew install z3
```

```
# Verify
z3 --version
# Z3 version 4.13.x

# Point Spectre at a non-default binary
spectre verify --backend z3 --z3-bin /opt/z3/bin/z3 account.spec
```

Spectre looks for `z3` on `$PATH` by default.

18 Appendix A: Language Grammar (EBNF)

The complete formal grammar for Spectre. Use this as the authoritative reference when the book's examples conflict with your intuition about syntax.

Notation: - `::=` — definition - `|` — alternative - `[ X ]` — X is optional - `{ X }` — zero or more X - `( X | Y )` — grouping/choice - `"word"` — literal keyword or symbol

---

18.1 A.1 Top-level structure

A Spectre file is a sequence of declarations in any order.

```
File    ::= { TopDecl }

TopDecl ::= [ "description" STRING ]
            ( VarDecl
            | ConstDecl
            | ParamDecl
            | TypeAliasDecl
            | EnumDecl
            | InitDecl
            | ActionDecl
            | FunDecl
            | InvariantDecl
            | TemporalDecl
            | ModuleDecl
            | ImportDecl
            )
```

Example — a minimal file:

```
var x: int
init { x = 0 }
action step { x' = x + 1 }
invariant positive { x >= 0 }
```

18.2 A.2 Variable and constant declarations

```
VarDecl    ::= "var" Ident ":" TypeExpr

ConstDecl  ::= "const" Ident ":" TypeExpr "=" Expr

ParamDecl  ::= "param" Ident ":" TypeExpr
```

- `var` — a mutable state variable, changed by actions.
- `const` — a fixed value baked into the spec.
- `param` — a named constant whose value is supplied at run time via `--param NAME=value`.

```
var balance: int
const LIMIT: int = 1000
param NUM_NODES: int          // set with --param NUM_NODES=5
```

18.3 A.3 Types

```
TypeExpr ::= "int"
           | "bool"
           | "str"
           | "float"
           | "Set"  "<" TypeExpr ">"
           | "Map"  "<" TypeExpr "," TypeExpr ">"
           | "List" "<" TypeExpr ">"
           | "Option" "<" TypeExpr ">"
           | "{" FieldDecl { "," FieldDecl } "}"     (* record type *)
           | "(" TypeExpr { "," TypeExpr } ")"       (* tuple type *)
           | Ident                                   (* named type or enum *)

FieldDecl ::= Ident ":" TypeExpr
```

Examples:

```
var count:   int
var active:  bool
var nodes:   Set<int>
var votes:   Map<int, bool>
var history: List<str>
var maybe:   Option<int>
```

18.4 A.4 Enum declarations

```
EnumDecl    ::= "enum" Ident "{" EnumVariant { "," EnumVariant } "}"
EnumVariant ::= Ident
```

Variants are always referenced as `EnumName.Variant`.

```
enum Phase { Preparing, Committed, Aborted }
var phase: Phase
init { phase = Phase.Preparing }
```

18.5 A.5 Type aliases

```
TypeAliasDecl ::= "type" Ident "=" TypeExpr
```

```
type NodeId = int
type VoteMap = Map<NodeId, bool>
var votes: VoteMap
```

18.6 A.6 Init declaration

```
InitDecl  ::= "init" ( Block | OneOfBody | Expr )

Block     ::= "{" { AssignStmt } "}"

OneOfBody ::= "oneOf" "{" OneOfOpt { "," OneOfOpt } "}"
OneOfOpt  ::= Expr
            | "{" { Ident "=" Expr } "}"
```

- Single block: one deterministic initial state.
- `oneOf`: the verifier explores from every listed initial state.

```
// Deterministic
init { balance = 0  frozen = false }

// Non-deterministic: explore from two starting states
init oneOf {
  { balance = 0, frozen = false },
  { balance = 100, frozen = false }
}
```

18.7 A.7 Action declarations

```
ActionDecl ::= "action" Ident [ "(" ParamList ")" ] [ "when" Expr ] "{" { Stmt } "}"
ParamList  ::= Param { "," Param }
Param      ::= Ident ":" TypeExpr
```

The optional `when Expr` clause is a **declaration-level guard** — it is equivalent to placing `require Expr` as the first statement of the action body. The tutorial chapters use `require` exclusively; `when` is offered as a compact alternative for a single top-level precondition, most useful with parameterized actions.

```
// These two forms are equivalent:
action deposit(amount: int) when amount > 0 {
```

```
  require balance + amount <= LIMIT
  balance' = balance + amount
}

action deposit(amount: int) {
  require amount > 0
  require balance + amount <= LIMIT
  balance' = balance + amount
}
```

18.8 A.8 Statements

```
Stmt       ::= AssignStmt
             | RequireStmt
             | EnsureStmt
             | LetStmt
             | ReturnStmt
             | ExprStmt

AssignStmt ::= Ident "'" "=" Expr     (* next-state assignment *)

RequireStmt ::= "require" Expr        (* precondition - action disabled if false *)
EnsureStmt  ::= "ensure" Expr         (* postcondition - invariant on next state *)
LetStmt     ::= "let" Ident "=" Expr  (* local name binding *)
ReturnStmt  ::= "return" Expr         (* return value from fun *)
ExprStmt    ::= Expr
```

The prime `'` in `x' = e` marks a **next-state assignment**. All right-hand sides are evaluated in the pre-action state (see Appendix B).

```
action transfer(amount: int) {
  require balance >= amount     // guard: evaluated in current state
  balance' = balance - amount   // next-state assignment
}
```

18.9 A.9 Function declarations

```
FunDecl ::= "fun" Ident "(" [ ParamList ] ")" [ ":" TypeExpr ] "{" { Stmt } "}"
```

Functions are pure — no side effects, no state mutation. They can be called from expressions.

```
fun clamp(v: int, lo: int, hi: int): int {
  if v < lo then lo else if v > hi then hi else v
}
```

18.10 A.10 Invariant and temporal declarations

```
InvariantDecl ::= "invariant" Ident "{" Expr "}"

TemporalDecl  ::= "temporal" Ident "{" TemporalExpr "}"
```

```
invariant solvent { balance >= 0 }
temporal  progress { WF(deposit) }
```

18.11 A.11 Expressions

```
Expr ::= Expr BinOp Expr       (* binary operation *)
       | UnaryOp Expr           (* unary prefix *)
       | Expr "." Ident         (* field / method access *)
       | Expr "(" [ ArgList ] ")"  (* method or function call *)
       | Expr "[" Expr "]"      (* index access *)
       | Ident [ "'" ]          (* variable; primed = next-state read *)
       | INT | FLOAT | STRING | BOOL
       | "(" Expr ")"
       | IfExpr
       | LetExpr
       | LambdaExpr
       | SetLiteral
       | ListLiteral
       | RecordLiteral

BinOp    ::= "+" | "-" | "*" | "/" | "%"
           | "==" | "!=" | "<" | "<=" | ">" | ">="
           | "&&" | "||"
           | "in"

UnaryOp  ::= "!"          (* logical NOT *)
           | "-"          (* arithmetic negation *)

ArgList  ::= Expr { "," Expr }
```

The `in` operator checks membership: `x in S` is `true` if `x` is an element of set `S` (or a key in map `S`).

```
invariant has_quorum { quorum_nodes.size() > NUM_NODES / 2 }
invariant leader_voted { leader in voted_set }
```

18.12 A.12 If, let, and lambda expressions

```
IfExpr     ::= "if" Expr "then" Expr "else" Expr
             | "if" "(" Expr ")" "{" Expr "}" "else" Expr
```

```
LetExpr    ::= "let" Ident "=" Expr

LambdaExpr ::= Ident "=>" Expr
             | "(" ParamList ")" "=>" Expr
```

```
let capped = if balance > LIMIT then LIMIT else balance
let double = x => x * 2
```

18.13 A.13 Collection literals

```
SetLiteral    ::= "{" "}"
                | "{" Expr { "," Expr } "}"

ListLiteral   ::= "[" "]"
                | "[" Expr { "," Expr } "]"

RecordLiteral ::= "{" FieldInit { "," FieldInit } [ "," "..." Ident ] "}"
FieldInit     ::= Ident ":" Expr
```

There is no `MapLiteral` syntax. Maps are always initialized to `{}` (empty set literal) or built up via `.put(k, v)` calls. The parser disambiguates `{}` by the declared type of the variable being assigned: when the target type is `Map<K,V>`, the empty brace literal `{}` is interpreted as an empty map; when the target type is `Set<T>`, it is an empty set. Non-empty set literals `{e1, e2, ...}` are unambiguously sets — there is no key-value literal syntax for maps.

```
var nodes:   Set<int>       = {1, 2, 3}  // SetLiteral (non-empty)
var history: List<str>      = ["start"]   // ListLiteral
var empty_s: Set<int>       = {}          // empty set (type drives interpretation)
var empty_m: Map<int, bool> = {}          // empty map (same syntax, type-driven)
```

18.14 A.14 Temporal expressions

```
TemporalExpr ::= "always"      "(" TemporalExpr ")"
               | "eventually"  "(" TemporalExpr ")"
               | "next"        "(" TemporalExpr ")"
               | "WF"          "(" Ident ")"
               | "SF"          "(" Ident ")"
               | TemporalExpr "until" TemporalExpr
               | TemporalExpr ( "->" | "~>" ) TemporalExpr
               | TemporalExpr "&&" TemporalExpr
               | TemporalExpr "||" TemporalExpr
               | "(" TemporalExpr ")"
               | Expr                                  (* state predicate *)
```

Operator	Meaning
`always(P)`	P holds in every state of every execution

Operator	Meaning
`eventually(P)`	P holds in some future state
`P until Q`	P holds until Q becomes true
`P -> Q`	Leads-to: whenever P holds, Q eventually holds
`WF(action)`	Weak fairness: if action is continually enabled, it fires
`SF(action)`	Strong fairness: if action is infinitely often enabled, it fires

```
temporal progress    { WF(increment) }
temporal completion  { eventually(phase == Phase.Committed) }
temporal stability   { always(balance >= 0) }
```

18.15 A.15 Module system

```
ModuleDecl ::= "module" Ident [ "extends" Ident ] "{" { TopDecl } "}"
             | "module" Ident "=" Ident "with" "{" { Ident "=" Expr } "}"

ImportDecl ::= "import" Ident
```

- `extends` copies all declarations from the parent into the child scope (macro substitution — not runtime inheritance).
- `with` instantiates a parameterized module with concrete values.
- `import` brings a module name into scope so its members can be accessed as `ModuleName.member`.

```
module Semaphore {
  var slots: int
  const MAX_SLOTS: int = 3
  action acquire { require slots > 0  slots' = slots - 1 }
  action release { require slots < MAX_SLOTS  slots' = slots + 1 }
  invariant valid { slots >= 0 && slots <= MAX_SLOTS }
}

module Mutex extends Semaphore {
  const MAX_SLOTS: int = 1    // override: binary semaphore
}
```

18.16 A.16 Operator precedence

From lowest (loosest) to highest (tightest):

Level	Operators	Associates
1 (loosest)	`||` (or)	left
2	`&&` (and)	left

Level	Operators	Associates
3	`== != in`	left
4	`< <= > >=`	left
5	`+ -`	left
6	`* / %`	left
7	`!` `-` (unary)	prefix
8	`.` (member) `(` (call)	left
9 (tightest)	`[` (index)	left

When in doubt, add parentheses — the parser accepts them freely.

18.17 A.17 Comments

```
LineComment  ::= "//" { any_char } newline
BlockComment ::= "/*" { any_char } "*/"
```

Comments have no effect on semantics and are stripped before parsing.

19 Appendix B: Evaluation Order and Priming Rules

Two questions come up constantly when writing Spectre specs:

1. **Priming:** "If I write `x' = y` and `y' = x` in the same action, do I get a swap or an infinite loop?"
2. **Expressions:** "What order are sub-expressions evaluated in, and can they affect state?"

This appendix answers both definitively.

19.1 B.1 Simultaneous assignment

Spectre actions use **simultaneous assignment** — the semantics of TLA+ actions.

The rule:

> Every right-hand side in an action is evaluated in the **pre-action state** (the state before the action fires). No assignment takes effect until all right-hand sides have been evaluated.

```
action swap {
  x' = y    // reads y from pre-action state
  y' = x    // reads x from pre-action state
}
```

After `swap` fires, `x` has the old value of `y` and `y` has the old value of `x`. This is a correct swap — there is no "temporary variable" needed.

Compare with an imperative language:

```
// Imperative (wrong for swap)
x = y    // x is now y's old value
y = x    // y is now x's NEW value (= y's old value) - no swap!

// Spectre (correct swap)
x' = y   // x_next = y_old
y' = x   // y_next = x_old   <- reads x_old, not x_next
```

19.2 B.2 Implicit UNCHANGED

Variables not assigned in an action keep their values automatically.

```
var balance: int
var frozen:  bool

action deposit(amount: int) {
  require !frozen
  balance' = balance + amount
  // frozen is NOT assigned - implicitly: frozen' = frozen
}
```

You never need to write `frozen' = frozen` manually. Any variable with no `x'` assignment in an action is implicitly unchanged.

19.3 B.3 Double assignment is an error

Each variable may be assigned at most once per action. Assigning the same variable twice is a parse error:

```
// ILLEGAL - balance' assigned twice
action bad {
  balance' = balance + 10
  balance' = balance + 20   // parse error
}
```

If you need conditional assignment, use `if`:

```
action conditional(amount: int) {
  balance' = if amount > 0 then balance + amount else balance
}
```

19.4 B.4 Guards evaluate in the pre-action state

`require` and `ensure` are evaluated in the same pre-action state as all other right-hand sides.

```
action withdraw(amount: int) {
  require balance >= amount   // reads balance BEFORE withdrawal
```

```
  balance' = balance - amount
}
```

If `require` evaluates to `false`, the action is **disabled** — it simply does not fire in that state. The verifier tries the next enabled action.

`ensure` is a postcondition: it is checked in the **post-action state** (after all assignments take effect).

```
action increment {
  x' = x + 1
  ensure x' > 0   // checked in next state: x + 1 > 0
}
```

19.5 B.5 Init is also simultaneous

`init` bodies follow the same simultaneous-assignment rule:

```
init {
  x = 10
  y = x    // y = 10 (reads x from the init state being built, NOT from x=10 above)
}
```

In practice, write `init` assignments as independent constant expressions — this avoids confusion and is consistent with how the verifier builds the initial state.

19.6 B.6 Actions are atomic

An action fires as a single indivisible step. There is no "intermediate state" between the first and last assignment. The transition relation maps the pre-action state directly to the post-action state.

```
pre-state s
    |
    | action fires (all RHS evaluated in s, all LHS applied simultaneously)
    |
    v
post-state s'
```

This matches TLA+'s action semantics exactly.

19.7 B.7 Expression evaluation

19.7.1 Purity

Expressions in Spectre are **pure**: they produce a value and have no side effects. No expression can modify state. Only `x' = e` assignments (in action and init bodies) change state.

19.7.2 Arithmetic

`int` is **unbounded**: there is no overflow, no wrapping, no undefined behavior. `+`, `-`, `*` are exact.

```
invariant no_overflow { x + y < 1000000 }  // always checkable, never wraps
```

`/` is integer division (truncates toward zero). `%` is the remainder (sign matches dividend).

19.7.3 Short-circuit evaluation

`&&` and `||` short-circuit left-to-right:

- `false && e` -> `false` without evaluating `e`
- `true || e` -> `true` without evaluating `e`

This matters when `e` is a method call that might fail on an empty collection:

```
// Safe: if nodes is empty, nodes.head() is never called
invariant safe { nodes.size() > 0 && nodes.head() > 0 }
```

19.7.4 The `in` operator

`x in S` checks whether `x` is an element of set `S` (or a key in map `S`). It evaluates `S` first, then checks membership.

```
invariant leader_valid { leader in active_nodes }
```

For maps, `k in m` returns `true` if key `k` has an entry in map `m`.

19.7.5 Method calls

`obj.method(args)` evaluates `obj`, then each argument left-to-right, then applies the method. Methods return a new value — they never mutate `obj` in place.

```
// nodes is unchanged; union returns a new set
nodes' = nodes.union({new_node})
```

19.7.6 Collection method quick reference

Collection	Methods
`Set<T>`	`.union(S)`, `.intersection(S)`, `.difference(S)`, `.contains(x)`, `.filter(f)`, `.size()`, `.toList()`
`Map<K,V>`	`.put(k,v)`, `.get(k)`, `.contains(k)`, `.size()`
`List<T>`	`.append(x)`, `.head()`, `.tail()`, `.filter(f)`, `.map(f)`, `.size()`, `.toSet()`

19.7.7 Collection error semantics

Two methods can fail at runtime if called on an ill-formed value:

- `M.get(k)` — **disables the action** if key `k` is not present in map `M`. The action is treated as if `require false` were its first statement — it cannot fire from any state where the key is missing. This is the same behavior as a failed `require`.
- `L.head()` — **disables the action** if list `L` is empty. Same semantics as `require !L.isEmpty()`.

The rationale: a Spectre action either succeeds atomically or is never attempted. There is no concept of a "partial action" or a "runtime panic." If a method call would fail, the action is simply not enabled in that state.

This matters for verification: if your spec has an action that calls `M.get(k)` on a key that may be absent, BFS/TLC will silently skip that action in states where the key is missing. Add a defensive `require k in M` guard before `M.get(k)` if you want to make the guard explicit and visible in traces.

```
action apply_vote {
  require node_id in votes          // explicit guard - action disabled if missing
  let decision = votes.get(node_id) // safe: key guaranteed present
  ...
}
```

19.8 B.8 Priming summary

Syntax	Meaning	Where allowed
`x`	Current (pre-action) value of variable `x`	Anywhere in expressions
`x'` on LHS	Next-state assignment: set `x` in post-state	Left-hand side of `AssignStmt` only
`x'` on RHS	Next-state read: read `x` from post-state	Allowed inside `ensure` only
Unassigned `x`	Implicit `x' = x`	Variables with no assignment in an action

The golden rule: write `x' = f(x, y, z)` where `x`, `y`, `z` are all pre-action values. If you need to refer to a computed value multiple times, use `let`:

```
action update(delta: int) {
  let new_val = balance + delta
  require new_val >= 0
  require new_val <= LIMIT
  balance' = new_val
}
```